

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



SOICT

BÁO CÁO ĐỒ ÁN
HỆ THỐNG GIÁM SÁT VÀ TỐI ƯU HÓA
LOGISTICS THEO THỜI GIAN THỰC

GVHD: T.S. Trần Việt Trung

Nhóm 5:

Họ và tên	MSSV
Vũ Đình Hoàn	20239751
Vũ Đức Trọng	20235234
Hà Kim Minh	20220119
Thân Văn Long Nhật	20215626
Nguyễn Tiến Quang	20235204

Hà Nội, 5/2026

Mục lục

1	GIỚI THIỆU BÀI TOÁN	5
1.1	Giới thiệu	5
1.2	Vấn đề	5
1.3	Mục tiêu dự án	6
1.4	Phạm vi và giới hạn của dự án	7
1.5	Định hướng tiếp cận	7
2	NGUỒN DỮ LIỆU VÀ CẤU TRÚC	9
2.1	Nguồn dữ liệu	9
2.2	Cấu trúc bản ghi GPS	9
2.3	Cấu trúc dữ liệu bản đồ	11
2.4	Cấu trúc traffic state trong Redis	13
2.5	Cấu trúc dữ liệu tuyến đường trong MongoDB	15
2.6	Luồng dữ liệu giữa các thành phần	17
3	KIẾN TRÚC HỆ THỐNG	19
3.1	Tổng quan kiến trúc	19
3.2	Kiến trúc theo tầng	20
3.3	Các thành phần hạ tầng	21
3.3.1	Kafka và Zookeeper	22
3.3.2	Redis	22
3.3.3	MongoDB	22
3.4	Các microservice chính	23
3.4.1	Bot simulator	24
3.4.2	Stream processing	25
3.4.3	Route optimization service	25
3.4.4	Dashboard backend	25

3.4.5	Dashboard frontend	26
3.5	Luồng xử lý end-to-end	26
3.6	Giao tiếp giữa các service	28
3.7	Triển khai bằng Kubernetes	29
3.8	Đặc điểm và giới hạn kiến trúc	30
4	XỬ LÝ DỮ LIỆU STREAMING BẰNG SPARK	32
4.1	Vai trò của tầng stream processing	32
4.2	Cấu hình Spark Structured Streaming	32
4.3	Đọc dữ liệu GPS từ Kafka	33
4.4	Map-matching GPS sang cạnh đường	35
4.5	Watermark, window và aggregate	36
4.6	Ghi traffic state vào Redis	37
4.7	Ghi dữ liệu GPS ra Parquet	39
4.8	Triển khai stream processing trong Kubernetes	39
4.9	Nhận xét và giới hạn	40
5	TỐI ƯU HÓA TUYẾN ĐƯỜNG	42
5.1	Vai trò của route optimization	42
5.2	Dữ liệu đầu vào của optimizer	42
5.3	Điều kiện kích hoạt tối ưu lại tuyến	43
5.4	Genetic Algorithm cho thứ tự giao khách	44
5.5	Hàm fitness và chi phí di chuyển	45
5.6	Selection, crossover và mutation	46
5.7	Sinh tuyến cạnh mới bằng Dijkstra trên graph	47
5.8	Lưu kết quả tối ưu vào MongoDB	48
5.9	Worker chạy nền trong Kubernetes	49
5.10	Nhận xét và giới hạn	51
6	KẾT QUẢ THỬ NGHIỆM VÀ ĐÁNH GIÁ	53

6.1	Mục tiêu thử nghiệm	53
6.2	Môi trường triển khai	53
6.3	Kết quả triển khai các service	54
6.4	Kiểm chứng luồng dữ liệu GPS	55
6.5	Kiểm chứng xử lý Spark và Redis	56
6.6	Kiểm chứng route optimization	57
6.7	Kết quả hiển thị trên dashboard	59
6.8	Đánh giá mức độ hoàn thành	61
6.9	Giới hạn trong thử nghiệm	63
6.10	Kết luận chương	63
7	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	65
7.1	Tổng kết dự án	65
7.2	Các kết quả đạt được	65
7.3	Bài học kinh nghiệm	66
7.4	Hạn chế của hệ thống	67
7.5	Hướng phát triển	69
7.6	Kết luận chung	70

Danh sách hình vẽ

3.1	Kiến trúc tổng thể của hệ thống logistics gần thời gian thực	20
3.2	Dashboard backend kết nối Kafka, Redis, MongoDB và sẵn sàng phát dữ liệu tới giao diện.	26
3.3	Trạng thái các thành phần hệ thống khi triển khai trên Kubernetes	29
4.1	Bot simulator sinh dữ liệu GPS và gửi vào Kafka topic <code>gps_stream</code>	35
4.2	Một số bản ghi GPS được truyền qua Kafka topic <code>gps_stream</code> .	35
4.3	Spark Structured Streaming xử lý micro-batch và ghi trạng thái giao thông vào Redis	40
5.1	Dữ liệu tuyến đường, khách hàng và lý do reroute trong collection <code>assigned_routes</code>	49
5.2	Route optimization worker tải graph đường và cập nhật lại tuyến cho các xe khi có tắc nghẽn	50
5.3	Thông tin khách hàng và tuyến đường mới được lưu trong MongoDB sau quá trình tối ưu	50
5.4	Trạng thái tuyến đường được đánh dấu tối ưu cùng thời điểm cập nhật trong MongoDB	51
6.1	Dashboard hiển thị tuyến đường được chọn cùng ETA và quãng đường còn lại	60
6.2	Thông tin phương tiện và tuyến đường hiển thị trực tiếp trên bản đồ	61

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN

1.1. Giới thiệu

Trong các hệ thống logistics hiện đại, việc vận chuyển hàng hóa không còn chỉ là bài toán tìm một tuyến đường ngắn nhất trên bản đồ tĩnh. Trên thực tế, một tuyến đường hợp lý tại thời điểm xuất phát có thể nhanh chóng trở nên kém hiệu quả khi tình trạng giao thông thay đổi, mật độ phương tiện tăng cao hoặc một đoạn đường bất ngờ bị ùn tắc. Điều này khiến bài toán điều phối đội xe trở thành một bài toán động, trong đó dữ liệu vị trí, tốc độ và trạng thái giao thông cần được cập nhật liên tục theo thời gian thực.

Với một đội xe giao hàng hoạt động trong khu vực đô thị, mỗi phương tiện liên tục sinh ra dữ liệu GPS gồm tọa độ, vận tốc và thời điểm ghi nhận. Khi số lượng phương tiện tăng lên, luồng dữ liệu này trở thành một dòng sự kiện lớn, đòi hỏi hệ thống phải có khả năng tiếp nhận, xử lý, lưu trữ trạng thái và phản hồi trong thời gian ngắn. Đây chính là bối cảnh phù hợp để áp dụng các công nghệ xử lý dữ liệu lớn như Apache Kafka, Spark Structured Streaming, Redis và MongoDB.

Dự án **Hệ thống điều phối vận tải thời gian thực** xây dựng một hệ thống giám sát và hỗ trợ tối ưu hóa logistics gần thời gian thực. Hệ thống mô phỏng luồng dữ liệu GPS từ nhiều phương tiện, đưa dữ liệu vào Kafka, xử lý trạng thái giao thông bằng Spark Structured Streaming, lưu traffic state theo từng cạnh đường trong Redis, hỗ trợ tái tối ưu thứ tự giao khách và cập nhật tuyến đường dựa trên tình trạng giao thông hiện tại, sau đó hiển thị kết quả trên dashboard.

1.2. Vấn đề

Bài toán logistics gần thời gian thực đặt ra nhiều thách thức ở cả tầng dữ liệu và tầng điều phối. Trước hết, dữ liệu GPS được sinh liên tục từ nhiều phương tiện, có tần suất cập nhật cao và cần được xử lý trong thời gian ngắn. Nếu hệ thống chỉ lưu dữ liệu mà không xử lý streaming, thông tin giao thông sẽ nhanh chóng mất giá trị khi so với trạng thái thực tế trên đường.

Thứ hai, dữ liệu GPS thô chưa trực tiếp phản ánh tình trạng giao thông. Một điểm tọa độ cần được ánh xạ vào đoạn đường tương ứng trên bản đồ, hay còn gọi là map-matching. Sau bước này, hệ thống mới có thể tổng hợp vận tốc trung bình, số lượng phương tiện và thời gian di chuyển ước lượng trên từng cạnh đường.

Thứ ba, route assignment ban đầu có thể không còn phù hợp khi một số cạnh đường trở nên tắc nghẽn. Vì vậy, hệ thống cần có cơ chế phát hiện các cạnh đường có tốc độ thấp hoặc bị đánh dấu tắc, sau đó kích hoạt tối ưu lại tuyến giao hàng. Quá trình tối ưu không chỉ cần tìm thứ tự giao khách hợp lý, mà còn cần sinh ra tuyến đường ở mức cạnh đường để dashboard có thể hiển thị và theo dõi.

Cuối cùng, toàn bộ pipeline cần được triển khai dưới dạng nhiều service độc lập, có khả năng giao tiếp qua mạng, tái khởi động khi lỗi và quan sát trạng thái trong quá trình chạy. Đây là lý do dự án sử dụng Docker và Kubernetes để container hóa và triển khai các thành phần của hệ thống.

1.3. Mục tiêu dự án

Mục tiêu của dự án là xây dựng một hệ thống logistics gần thời gian thực theo định hướng Big Data, có khả năng thể hiện đầy đủ vòng đời của dữ liệu từ lúc được sinh ra, truyền qua hệ thống message broker, xử lý theo luồng, lưu trữ trạng thái, hỗ trợ tối ưu tuyến đường và trực quan hóa trên dashboard.

Các mục tiêu chính của dự án gồm:

- Mô phỏng luồng dữ liệu GPS từ nhiều phương tiện di chuyển trên bản đồ.
- Sử dụng Kafka làm tầng tiếp nhận và phân phối luồng sự kiện GPS.
- Sử dụng Spark Structured Streaming để xử lý dữ liệu gần thời gian thực, thực hiện map-matching và tổng hợp traffic theo từng cạnh đường.
- Lưu trạng thái giao thông ngắn hạn trong Redis để phục vụ truy vấn độ trễ thấp.
- Lưu thông tin tuyến đường và kết quả tối ưu trong MongoDB.
- Hỗ trợ tái tối ưu thứ tự giao khách và sinh tuyến đường mới dựa trên trạng thái giao thông hiện tại.

- Hiển thị vị trí xe, trạng thái giao thông và tuyến đường trên dashboard gần thời gian thực.
- Triển khai các thành phần bằng Docker và Kubernetes.

1.4. Phạm vi và giới hạn của dự án

Dự án tập trung xây dựng và kiểm chứng một pipeline dữ liệu lớn thời gian thực trong bài toán logistics. Dữ liệu GPS và tình trạng giao thông trong hệ thống được sinh bằng simulator để nhóm có thể chủ động kiểm soát tải, tần suất cập nhật và các kịch bản thử nghiệm. Trong phạm vi bài tập lớn, hệ thống chưa tích hợp trực tiếp với dữ liệu giao thông thực tế từ các dịch vụ bản đồ hoặc cảm biến ngoài đời.

Hệ thống được triển khai trong môi trường cục bộ bằng Docker và Kubernetes, do đó quy mô thử nghiệm còn bị giới hạn bởi tài nguyên máy cá nhân. Các thông số như số lượng phương tiện, tần suất gửi dữ liệu và kích thước bản đồ được lựa chọn để đảm bảo hệ thống có thể chạy ổn định trong điều kiện phần cứng giới hạn.

Phần route optimization được thiết kế như một thành phần tiêu thụ dữ liệu giao thông đã xử lý, qua đó kiểm chứng khả năng sử dụng traffic state để hỗ trợ tái tối ưu tuyến đường. Trong phạm vi môn Big Data, trọng tâm của đề tài không phải chứng minh tính tối ưu toàn cục của thuật toán định tuyến, mà là xây dựng được cơ chế tích hợp giữa stream processing, state storage, route update và dashboard gần thời gian thực trong một pipeline end-to-end.

1.5. Định hướng tiếp cận

Khác với các hệ thống chỉ phân tích dữ liệu sau khi đã lưu trữ, dự án lựa chọn hướng tiếp cận event-driven streaming. Dữ liệu GPS được xem như một chuỗi sự kiện liên tục. Kafka đảm nhận vai trò trung gian tiếp nhận và phân phối sự kiện; Spark Structured Streaming xử lý dữ liệu theo micro-batch; Redis giữ trạng thái giao thông mới nhất; MongoDB lưu các route assignment và optimization result; dashboard nhận dữ liệu gần thời gian thực thông qua Socket.IO.

Cách tiếp cận này giúp hệ thống phản ứng với thay đổi giao thông theo cơ chế thời gian thực, phù hợp hơn so với việc chỉ sử dụng route tĩnh. Đồng thời, kiến trúc

nhiều service cũng phản ánh đúng tinh thần của các hệ thống dữ liệu lớn hiện đại: tách biệt ingestion, processing, state management, optimization và visualization để dễ mở rộng, debug và triển khai.

CHƯƠNG 2. NGUỒN DỮ LIỆU VÀ CẤU TRÚC

2.1. Nguồn dữ liệu

Dữ liệu đầu vào của hệ thống được sinh bởi module mô phỏng phương tiện. Thay vì kết nối trực tiếp tới thiết bị GPS ngoài đời, dự án sử dụng simulator để tạo ra luồng sự kiện vị trí của nhiều xe đang di chuyển trên bản đồ. Cách tiếp cận này giúp nhóm chủ động kiểm soát số lượng phương tiện, tần suất sinh dữ liệu và các tình huống thử nghiệm như thay đổi tốc độ, ùn tắc hoặc cập nhật lại tuyến đường.

Các bản ghi GPS sau khi được sinh ra được gửi vào Kafka topic `gps_stream`. Kafka đóng vai trò là lớp tiếp nhận dữ liệu trung gian, giúp tách biệt module sinh dữ liệu với các thành phần xử lý phía sau. Spark Structured Streaming đọc dữ liệu từ Kafka, parse dữ liệu theo schema định nghĩa trước, thực hiện map-matching tọa độ GPS sang cạnh đường và tổng hợp trạng thái giao thông theo từng cạnh.

Bên cạnh luồng GPS, hệ thống còn sử dụng dữ liệu bản đồ để xây dựng đồ thị đường. Trong repository, dữ liệu bản đồ và schema cạnh đường được đặt trong thư mục `data/`, bao gồm các file như `hanoi_map.osm` và `edges_schema.json`. Dữ liệu này được dùng cho cả stream processing, nhằm ánh xạ tọa độ vào cạnh đường, và route optimization, nhằm sinh tuyến đường mới ở mức cạnh.

Về mặt lưu trữ, hệ thống sử dụng Redis cho trạng thái giao thông ngắn hạn và MongoDB cho dữ liệu tuyến đường. Redis lưu traffic state theo từng cạnh đường với TTL ngắn, phù hợp với dữ liệu có giá trị thời gian. MongoDB lưu các bản ghi route assignment và kết quả tối ưu, phục vụ dashboard và quá trình theo dõi thay đổi tuyến đường.

2.2. Cấu trúc bản ghi GPS

Mỗi bản ghi GPS biểu diễn trạng thái của một phương tiện tại một thời điểm cụ thể. Về bản chất, đây là một sự kiện thời gian thực, gồm định danh phương tiện, loại đối tượng, vị trí, vận tốc, cạnh đường hiện tại và thời điểm ghi nhận. Các bản ghi này được truyền qua Kafka ở dạng JSON để các thành phần phía sau có thể đọc

và xử lý theo micro-batch hoặc sự kiện gần thời gian thực.

Trường	Kiểu dữ liệu	Ý nghĩa
entity_id	String	Mã định danh của phương tiện, ví dụ Bot_0001 hoặc Truck_001.
entity_type	String	Loại đối tượng mô phỏng, gồm Bot hoặc Truck.
latitude	Double	Vĩ độ hiện tại của phương tiện.
longitude	Double	Kinh độ hiện tại của phương tiện.
speed	Double	Vận tốc hiện tại của phương tiện, đơn vị km/h.
edge_id	String	Cạnh đường hiện tại theo simulator. Field này hữu ích cho dashboard, nhưng Spark vẫn thực hiện map-matching lại từ tọa độ GPS.
timestamp	Long	Thời điểm sinh sự kiện, tính theo epoch milliseconds.
route_index	Integer	Chỉ có với Truck, biểu diễn vị trí hiện tại trong route edge sequence.
route_total	Integer	Chỉ có với Truck, biểu diễn tổng số cạnh trong route hiện tại.

Bảng 2.1. Cấu trúc bản ghi GPS thô được gửi vào Kafka topic `gps_stream`

Trong triển khai Spark hiện tại, schema xử lý streaming chỉ sử dụng các trường `entity_id`, `entity_type`, `latitude`, `longitude`, `speed` và `timestamp`. Trường `edge_id` trong message thô không được dùng trực tiếp cho bước tổng hợp traffic, mà hệ thống thực hiện map-matching lại từ tọa độ GPS để đảm bảo thống nhất với dữ liệu bản đồ.

```
{
  "entity_id": "Truck_001",
  "entity_type": "Truck",
  "latitude": 21.0278,
  "longitude": 105.8342,
  "speed": 28.5,
```

```
"edge_id": "E_123_456",  
"timestamp": 1716000000000,  
"route_index": 3,  
"route_total": 42  
}
```

Listing 2.1. Ví dụ bản ghi GPS được gửi vào Kafka

Dữ liệu GPS thô chưa thể được sử dụng trực tiếp để đánh giá trạng thái giao thông, vì một tọa độ riêng lẻ chưa đủ để xác định đầy đủ trạng thái của một đoạn đường. Do đó, bước map-matching là cần thiết để chuyển đổi tọa độ GPS thành `edge_id`. Sau khi có `edge_id`, hệ thống có thể nhóm dữ liệu theo từng cạnh đường để tính tốc độ trung bình, số lượng phương tiện và xác định cạnh có dấu hiệu tắc nghẽn.

2.3. Cấu trúc dữ liệu bản đồ

Dữ liệu bản đồ trong dự án được biểu diễn dưới dạng đồ thị đường. Mỗi đỉnh biểu diễn một điểm trong mạng lưới đường, còn mỗi cạnh biểu diễn một đoạn đường có hướng giữa hai điểm. Cách biểu diễn này phù hợp với cả hai nhu cầu: xử lý giao thông theo từng cạnh và tìm đường trong module route optimization.

File `hanoi_map.osm` được xử lý thành file `edges_schema.json`. Trong quá trình xử lý, hệ thống lọc các loại đường phù hợp cho phương tiện, đọc tên đường, tốc độ tối đa, hướng đường và tạo danh sách cạnh. Với đường hai chiều, hệ thống sinh hai cạnh tương ứng cho hai hướng di chuyển.

Trường	Kiểu dữ liệu	Ý nghĩa
edge_id	String	Mã định danh duy nhất của cạnh đường, có dạng E_<start>_<end>.
road_name	String	Tên tuyến đường, nếu dữ liệu OSM có cung cấp; nếu không có sẽ dùng giá trị mặc định.
max_speed_kmh	Double	Tốc độ tối đa hoặc tốc độ tham chiếu của cạnh đường, đơn vị km/h.
start_node	Object	Nút bắt đầu của cạnh, gồm node_id, lat, lon.
end_node	Object	Nút kết thúc của cạnh, gồm node_id, lat, lon.
length_meters	Double	Chiều dài cạnh đường, đơn vị mét, được tính từ tọa độ hai nút.

Bảng 2.2. Cấu trúc cạnh đường trong file edges_schema.json

```
{
  "edge_id": "E_123_456",
  "road_name": "Duong khong ten",
  "max_speed_kmh": 40.0,
  "start_node": {
    "node_id": "123",
    "lat": 21.0278,
    "lon": 105.8342
  },
  "end_node": {
    "node_id": "456",
    "lat": 21.0281,
    "lon": 105.8350
  },
  "length_meters": 92.4
}
```

Listing 2.2. Ví dụ một cạnh đường trong edges_schema.json

Trong pipeline streaming, dữ liệu bản đồ giúp Spark xác định `edge_id` tương ứng với mỗi bản ghi GPS. Trong module tối ưu tuyến đường, cùng dữ liệu này được sử dụng để xây dựng graph và tìm đường đi giữa các điểm giao hàng. Nhờ vậy, hệ thống có thể thống nhất giữa dữ liệu xử lý thời gian thực, dữ liệu tối ưu và dữ liệu hiển thị trên dashboard.

2.4. Cấu trúc traffic state trong Redis

Redis được sử dụng để lưu trạng thái giao thông ngắn hạn theo từng cạnh đường. Đây là tầng state có độ trễ thấp, phục vụ cho dashboard gần thời gian thực và hỗ trợ các quyết định reroute của hệ thống. Thay vì lưu toàn bộ lịch sử GPS, Redis chỉ giữ trạng thái mới nhất hoặc gần nhất của từng cạnh đường, giúp hệ thống phản hồi nhanh khi cần truy vấn tình trạng giao thông hiện tại.

Spark Structured Streaming tổng hợp dữ liệu theo cửa sổ thời gian, sau đó ghi kết quả vào Redis với key dạng `edge:<edge_id>`. Mỗi bản ghi traffic state có TTL ngắn để hạn chế việc sử dụng dữ liệu đã cũ. Ngoài ra, Redis duy trì set `blocked_edges`; nếu một cạnh có vận tốc trung bình thấp hơn ngưỡng tắc nghẽn, cạnh đó được thêm vào set này, ngược lại sẽ được loại bỏ.

Trường	Kiểu dữ liệu	Ý nghĩa
edge_id	String	Mã cạnh đường được tổng hợp từ dữ liệu GPS sau map-matching.
avg_speed	Double	Vận tốc trung bình của các phương tiện trên cạnh trong cửa sổ thời gian gần nhất.
vehicle_count	Integer	Số lượng phương tiện được ghi nhận trên cạnh trong window xử lý.
distance	Double	Chiều dài cạnh đường, đơn vị mét.
max_speed	Double	Tốc độ tham chiếu của cạnh đường, lấy từ dữ liệu bản đồ.
estimated_travel_time	Double	Thời gian di chuyển ước lượng qua cạnh đường, tính từ chiều dài cạnh và vận tốc trung bình.
is_congested	Boolean	Cờ đánh dấu cạnh đường đang tắc nghẽn, dựa trên ngưỡng vận tốc trung bình.
updated_at	String	Thời điểm cập nhật traffic state theo thời gian kết thúc window.
last_updated	Long	Thời điểm cập nhật dạng epoch milliseconds.

Bảng 2.3. Cấu trúc traffic state được lưu theo key edge:<edge_id> trong Redis

```
{
  "edge_id": "E_123_456",
  "avg_speed": 4.72,
  "vehicle_count": 38,
  "distance": 92.4,
  "max_speed": 40.0,
  "estimated_travel_time": 70.47,
  "is_congested": true,
  "updated_at": "2026-05-19T07:33:50",
```

```
"last_updated": 1779176030000
}
```

Listing 2.3. Ví dụ traffic state của một cạnh đường trong Redis

Cách tổ chức này phù hợp với đặc trưng của traffic state: dữ liệu cần được truy vấn nhanh, nhưng chỉ có giá trị trong một khoảng thời gian ngắn. Khi dashboard cần hiển thị trạng thái giao thông, backend có thể đọc các key `edge:*`. Khi cần phát hiện các cạnh có vấn đề, hệ thống có thể kiểm tra set `blocked_edges` thay vì quét toàn bộ dữ liệu.

2.5. Cấu trúc dữ liệu tuyến đường trong MongoDB

MongoDB được sử dụng để lưu dữ liệu tuyến đường được gán cho từng phương tiện và kết quả tối ưu hóa. Khác với Redis, dữ liệu trong MongoDB có tính bền vững hơn và được backend dashboard sử dụng để theo dõi route assignment, tuyến đường mới và kết quả tối ưu.

Dữ liệu ban đầu trong collection `assigned_routes` có thể được seed từ dữ liệu bản đồ. Mỗi bản ghi đại diện cho một xe tải, gồm thông tin định danh, cạnh đường hiện tại, tuyến đường đang được gán và danh sách khách hàng còn lại. Sau khi route optimization worker xử lý, bản ghi được cập nhật thêm các trường mô tả tuyến đường mới và kết quả tối ưu.

Trường	Kiểu dữ liệu	Ý nghĩa
vehicle_id	String	Mã định danh của xe được gán tuyến, ví dụ Truck_001.
entity_id	String	Mã định danh đối tượng, thường trùng với vehicle_id.
entity_type	String	Loại đối tượng, trong collection này chủ yếu là Truck.
edge_id	String	Cạnh đường hiện tại của xe tại thời điểm seed hoặc cập nhật.
current_edge_id	String	Cạnh đường hiện tại được optimizer dùng làm điểm bắt đầu khi tối ưu.
distance_on_edge	Double	Khoảng cách đã di chuyển trên cạnh hiện tại.
assigned_route	Array[String]	Tuyến đường đang được gán cho xe, biểu diễn bằng danh sách edge_id.
remaining_customers	Array[Object]	Danh sách điểm giao hàng còn lại, mỗi phần tử gồm cust_id, latitude, longitude.
new_assigned_route	Array[String]	Tuyến đường mới sau khi optimizer xử lý.
optimized_customer_order	Array	Thứ tự giao khách sau khi chạy Genetic Algorithm.
estimated_total_travel_time	Double	Tổng thời gian di chuyển ước lượng của route sau tối ưu.
optimization_result	Object	Thông tin chi tiết về quá trình tối ưu, gồm route cũ, route mới, chi phí, số thế hệ và thời điểm tối ưu.
last_optimized_at	Long	Thời điểm tối ưu gần nhất, dạng epoch milliseconds.
route_status	String	Trạng thái tuyến đường, ví dụ optimized.

Bảng 2.4. Cấu trúc collection assigned_routes trong MongoDB

```

{
  "vehicle_id": "Truck_001",
  "entity_id": "Truck_001",
  "entity_type": "Truck",
  "edge_id": "E_123_456",
  "current_edge_id": "E_123_456",
  "distance_on_edge": 0.0,
  "assigned_route": ["E_123_456", "E_456_789"],
  "remaining_customers": [
    {
      "cust_id": "Cust_T001_1",
      "latitude": 21.0278,
      "longitude": 105.8342
    }
  ],
  "new_assigned_route": ["E_123_456", "E_456_789"],
  "estimated_total_travel_time": 240.5,
  "route_status": "optimized"
}

```

Listing 2.4. Ví dụ bản ghi tuyến đường trong MongoDB

Route optimization service đọc dữ liệu từ collection `assigned_routes`, kết hợp với trạng thái tắc nghẽn trong Redis để quyết định có cần tối ưu lại hay không. Sau khi tối ưu, service cập nhật các trường như `new_assigned_route`, `optimized_customer_order`, `estimated_total_travel_time`, `optimization_result` và `route_status`. Backend dashboard theo dõi collection này bằng MongoDB Change Streams và đẩy cập nhật tuyến đường về frontend thông qua Socket.IO.

2.6. Luồng dữ liệu giữa các thành phần

Từ các cấu trúc dữ liệu trên, có thể tóm tắt luồng dữ liệu chính của hệ thống như sau: dữ liệu GPS được sinh bởi simulator, gửi vào Kafka, được xử lý bằng Spark Structured Streaming để tổng hợp trạng thái giao thông theo từng cạnh đường, lưu

traffic state vào Redis, kết hợp với dữ liệu route trong MongoDB để optimizer cập nhật tuyến đường khi cần, sau đó dashboard hiển thị trạng thái phương tiện, giao thông và tuyến đường.

1. Bot simulator sinh dữ liệu GPS và gửi vào Kafka topic `gps_stream`. Mỗi bản ghi có các thông tin như định danh phương tiện, tọa độ, vận tốc, thời điểm ghi nhận và `edge_id` hiện tại.
2. Spark Structured Streaming đọc dữ liệu từ Kafka, parse JSON theo schema định nghĩa trước, lọc các bản ghi hợp lệ và sử dụng `edge_id` trong message để nhóm dữ liệu theo cạnh đường.
3. Spark tổng hợp vận tốc trung bình và số lượng phương tiện theo từng `edge_id` trong cửa sổ thời gian gần nhất.
4. Kết quả traffic state được ghi vào Redis theo key `edge:<edge_id>`; các cạnh có dấu hiệu tắc nghẽn được cập nhật vào set `blocked_edges`.
5. Route optimization service đọc route assignment từ MongoDB và trạng thái giao thông từ Redis để xác định những tuyến có thể cần cập nhật.
6. Khi tuyến hiện tại bị ảnh hưởng bởi cạnh tắc nghẽn hoặc khi cần khởi tạo route tối ưu, optimizer chạy Genetic Algorithm để tối ưu thứ tự giao khách, sau đó sinh hoặc cập nhật `new_assigned_route` và các thông tin liên quan trong MongoDB.
7. Dashboard backend nhận vị trí phương tiện từ Kafka, đọc traffic state từ Redis và đọc route assignment từ MongoDB; các dữ liệu này được đẩy về frontend thông qua Socket.IO để hiển thị trên bản đồ.

Thiết kế này giúp mỗi tầng dữ liệu có vai trò rõ ràng: Kafka đảm nhiệm ingest sự kiện, Spark xử lý streaming và tổng hợp traffic theo cạnh đường, Redis lưu trạng thái giao thông ngắn hạn, MongoDB lưu dữ liệu tuyến đường và kết quả tối ưu, còn dashboard đảm nhiệm trực quan hóa. Đây là nền tảng để các chương sau trình bày sâu hơn về kiến trúc hệ thống, xử lý Spark và route optimization.

CHƯƠNG 3. KIẾN TRÚC HỆ THỐNG

3.1. Tổng quan kiến trúc

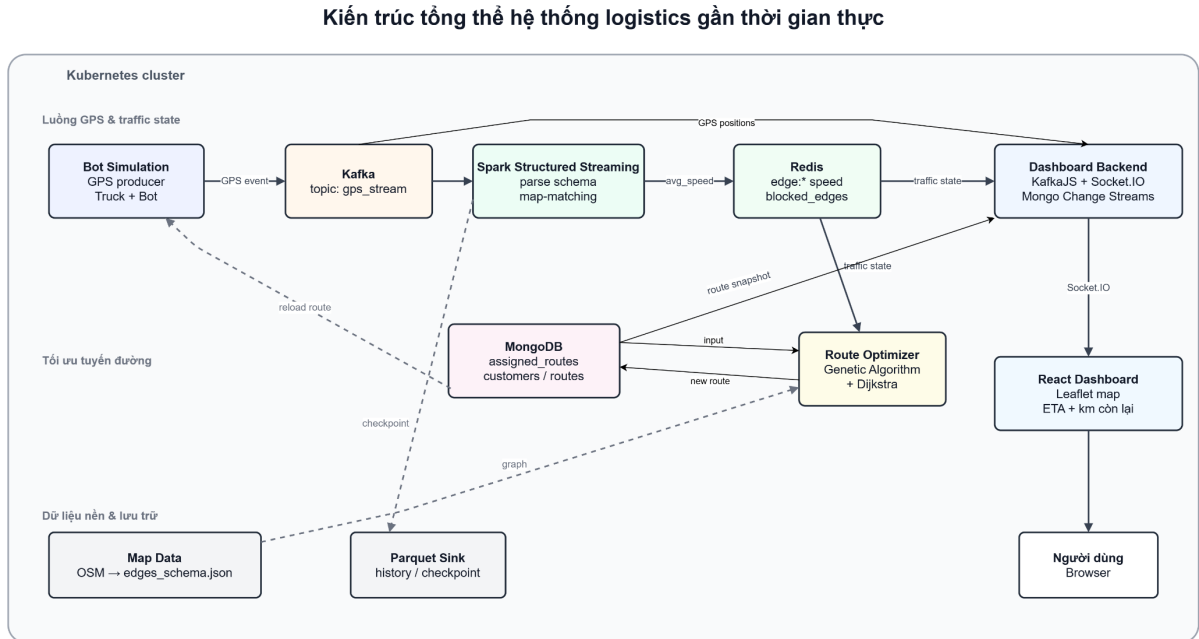
Hệ thống được thiết kế theo kiến trúc nhiều thành phần, trong đó mỗi service đảm nhiệm một vai trò riêng trong pipeline dữ liệu logistics gần thời gian thực. Dữ liệu GPS được sinh từ simulator, đưa vào Kafka, xử lý bằng Spark Structured Streaming, lưu trạng thái giao thông ngắn hạn trong Redis, lưu dữ liệu tuyến đường trong MongoDB và hiển thị lên dashboard thông qua backend gần thời gian thực.

Cách tổ chức này giúp hệ thống tách biệt rõ ràng giữa các nhiệm vụ: sinh dữ liệu, tiếp nhận sự kiện, xử lý streaming, quản lý trạng thái, tối ưu tuyến đường và trực quan hóa. Mỗi thành phần có thể được build thành Docker image riêng và triển khai dưới dạng Kubernetes Deployment. Điều này phù hợp với mục tiêu của bài tập lớn: không chỉ xây dựng thuật toán đơn lẻ, mà còn kiểm chứng một hệ thống dữ liệu lớn có khả năng vận hành theo pipeline end-to-end.

Về tổng thể, kiến trúc hệ thống có thể được mô tả theo luồng sau:

1. **Bot simulator** mô phỏng phương tiện và gửi bản ghi GPS vào Kafka topic `gps_stream`.
2. **Kafka** đóng vai trò message broker, tách biệt producer và các consumer phía sau.
3. **Spark Structured Streaming** đọc dữ liệu từ Kafka, parse JSON, thực hiện map-matching và tổng hợp traffic theo từng cạnh đường.
4. **Redis** lưu traffic state ngắn hạn theo key `edge:<edge_id>` và duy trì tập `blocked_edges`.
5. **Route optimization service** đọc route assignment từ MongoDB, kết hợp trạng thái tắc nghẽn từ Redis và cập nhật kết quả tối ưu trở lại MongoDB.
6. **Dashboard backend** đọc dữ liệu từ Kafka, Redis, MongoDB và đẩy dữ liệu gần thời gian thực tới frontend qua Socket.IO.
7. **Dashboard frontend** hiển thị vị trí phương tiện, trạng thái giao thông và

tuyến đường trên bản đồ.



Hình 3.1. Kiến trúc tổng thể của hệ thống logistics gần thời gian thực

Hình 3.1 Mô tả luồng dữ liệu chính của hệ thống. Bot Simulation sinh bản ghi GPS và gửi vào Kafka topic `gps_stream`. Spark Structured Streaming tiêu thụ luồng này để parse dữ liệu, map-matching lên đồ thị đường và cập nhật trạng thái giao thông vào Redis. Song song, Dashboard Backend cũng tiêu thụ vị trí phương tiện từ Kafka để phát dữ liệu gần thời gian thực tới giao diện qua Socket.IO. Route Optimizer đọc trạng thái giao thông từ Redis và dữ liệu tuyến đường trong MongoDB, sau đó ghi tuyến mới vào collection `assigned_routes`. Dashboard nhận dữ liệu vị trí, trạng thái giao thông và tuyến đường để hiển thị bản đồ, ETA và quãng đường còn lại.

3.2. Kiến trúc theo tầng

Để dễ phân tích, hệ thống có thể được chia thành năm tầng chính: tầng sinh dữ liệu, tầng truyền sự kiện, tầng xử lý streaming, tầng lưu trữ trạng thái và tầng hiển thị. Việc phân tầng giúp giảm phụ thuộc trực tiếp giữa các module và làm rõ trách nhiệm của từng thành phần.

Tầng	Thành phần	Vai trò
Sinh dữ liệu	bot-simulator	Mô phỏng phương tiện, tính vị trí hiện tại, vận tốc và gửi dữ liệu GPS vào Kafka.
Truyền sự kiện	Kafka, Zookeeper	Tiếp nhận và phân phối message GPS thông qua topic <code>gps_stream</code> .
Xử lý streaming	stream-processing	Đọc dữ liệu Kafka bằng Spark Structured Streaming, map-matching tọa độ GPS sang cạnh đường và tổng hợp traffic state.
Lưu trạng thái và dữ liệu	Redis, MongoDB	Redis lưu traffic state ngắn hạn; MongoDB lưu route assignment và kết quả tối ưu tuyến đường.
Tối ưu và hiển thị	route-optimization, dashboard-backend, dashboard-frontend	Hỗ trợ tái tối ưu tuyến đường khi phát hiện tắc nghẽn và hiển thị kết quả trên dashboard gần thời gian thực.

Bảng 3.1. Các tầng chính trong kiến trúc hệ thống

Tầng sinh dữ liệu và tầng xử lý streaming không giao tiếp trực tiếp với nhau, mà được nối qua Kafka. Điều này giúp Spark có thể đọc dữ liệu theo tốc độ phù hợp mà không làm chậm simulator. Tương tự, Redis và MongoDB được tách thành hai tầng lưu trữ với mục đích khác nhau: Redis ưu tiên truy vấn nhanh với dữ liệu ngắn hạn, còn MongoDB lưu cấu trúc tuyến đường và kết quả tối ưu có tính bền vững hơn.

3.3. Các thành phần hạ tầng

Các thành phần hạ tầng của hệ thống gồm Zookeeper, Kafka, Redis và MongoDB. Đây là các service nền được triển khai trước trong Kubernetes để các microservice phía trên có thể kết nối thông qua service name nội bộ.

3.3.1. Kafka và Zookeeper

Kafka được sử dụng làm message broker trung tâm cho luồng sự kiện GPS. Bot simulator đóng vai trò producer và gửi dữ liệu vào topic `gps_stream`. Spark Structured Streaming và dashboard backend đóng vai trò consumer, đọc dữ liệu từ Kafka để phục vụ các mục tiêu khác nhau.

Trong triển khai Kubernetes, Kafka chạy cùng Zookeeper. Zookeeper cung cấp cơ chế điều phối cần thiết cho Kafka trong cấu hình sử dụng image `confluentinc/cp-kafka:7.3.0`. Kafka được quảng bá qua địa chỉ nội bộ `kafka.default.svc.cluster.local:9092`, giúp các pod khác trong cluster có thể kết nối ổn định bằng service DNS.

3.3.2. Redis

Redis được sử dụng như một tầng lưu trạng thái giao thông ngắn hạn. Sau khi Spark tổng hợp dữ liệu theo cửa sổ thời gian, kết quả được ghi vào Redis theo từng cạnh đường. Các key có dạng `edge:<edge_id>` và chứa các thông tin như vận tốc trung bình, số lượng phương tiện, chiều dài cạnh và thời gian di chuyển ước lượng.

Redis phù hợp với vai trò này vì dữ liệu traffic state thay đổi liên tục và cần được truy vấn nhanh bởi dashboard hoặc route optimization service. Bên cạnh các key theo từng cạnh đường, hệ thống cũng duy trì tập `blocked_edges` để lưu các cạnh đang được đánh giá là tắc nghẽn theo ngưỡng vận tốc. Tập này giúp optimizer kiểm tra nhanh xem tuyến hiện tại có đi qua cạnh bất lợi hay không. Trên dashboard, trạng thái giao thông được biểu diễn trực quan bằng màu sắc của đoạn đường, ví dụ xanh cho thông thoáng, cam cho di chuyển chậm và đỏ cho tắc nghẽn.

3.3.3. MongoDB

MongoDB được sử dụng để lưu dữ liệu tuyến đường và kết quả tối ưu. Collection chính trong hệ thống là `assigned_routes`, chứa thông tin xe, tuyến đường đang được gán, danh sách khách hàng còn lại, tuyến mới sau tối ưu và các metadata liên quan đến quá trình tối ưu.

Trong Kubernetes, MongoDB được khởi động với replica set `rs0`. Cấu hình này cần thiết để backend có thể sử dụng MongoDB Change Streams, qua đó phát

hiện các thay đổi trong collection `assigned_routes` và đẩy sự kiện tuyến đường mới về dashboard. Do dự án chạy trong môi trường triển khai cục bộ, MongoDB hiện được triển khai với một replica.

3.4. Các microservice chính

Bên trên các service hạ tầng, hệ thống gồm năm microservice chính: bot simulator, stream processing, route optimization, dashboard backend và dashboard frontend. Mỗi service được đóng gói bằng Docker image riêng và triển khai bằng Kubernetes Deployment.

Deployment	Image	Vai trò
bot-simulator	logictics-bot:latest	Sinh dữ liệu GPS cho các phương tiện mô phỏng và gửi message vào Kafka topic <code>gps_stream</code> .
stream-processing	logictics-stream:v5	Chạy Spark Structured Streaming để đọc dữ liệu từ Kafka, parse JSON, tổng hợp <code>avg_speed</code> , <code>vehicle_count</code> theo <code>edge_id</code> và ghi traffic state vào Redis.
route-optimization	logictics-opt:v6	Đọc route assignment từ MongoDB và trạng thái giao thông từ Redis, sau đó cập nhật <code>new_assigned_route</code> khi cần tối ưu lại tuyến đường.
dashboard-backend	logictics-backend:v7	Kết nối Kafka, Redis và MongoDB; tổng hợp dữ liệu vị trí, traffic state và route update; phát dữ liệu gần thời gian thực cho frontend qua Socket.IO.
dashboard-frontend	logictics-frontend:v4	Giao diện bản đồ gần thời gian thực để hiển thị phương tiện, trạng thái giao thông, tuyến đường, ETA và quãng đường còn lại.

Bảng 3.2. Các microservice chính trong hệ thống

3.4.1. Bot simulator

Bot simulator là nguồn sinh dữ liệu chính của hệ thống. Service này đọc dữ liệu cạnh đường từ `edges_schema.json`, khởi tạo tập phương tiện mô phỏng và định kỳ gửi bản ghi GPS vào Kafka. Trong code hiện tại, simulator khởi tạo cả Bot và Truck; riêng Truck có route edge sequence và thông tin tiến độ route như `route_index`, `route_total`.

Việc mô phỏng số lượng lớn phương tiện tạo ra áp lực dữ liệu đủ để kiểm thử

pipeline streaming. Đồng thời, dữ liệu này vẫn có cấu trúc kiểm soát được, giúp nhóm dễ tái hiện lỗi, đánh giá luồng xử lý và quan sát kết quả thử nghiệm.

3.4.2. Stream processing

Stream processing service là thành phần xử lý dữ liệu lớn chính của hệ thống. Service này sử dụng Spark Structured Streaming để đọc topic `gps_stream`, parse dữ liệu JSON theo schema, lọc các bản ghi hợp lệ và chuyển timestamp dạng epoch milliseconds sang kiểu thời gian xử lý được bởi Spark.

Sau khi parse dữ liệu, Spark thực hiện map-matching để ánh xạ tọa độ GPS vào cạnh đường gần nhất. Dữ liệu sau đó được tổng hợp theo window với watermark để tạo traffic state theo từng cạnh. Kết quả tổng hợp được ghi vào Redis thông qua cơ chế `foreachBatch`, giúp Redis luôn giữ trạng thái giao thông mới nhất trong khoảng thời gian gần đây.

3.4.3. Route optimization service

Route optimization service đảm nhiệm việc cập nhật tuyến đường khi phát hiện điều kiện giao thông bất lợi. Service này đọc các document trong collection `assigned_routes`, lấy danh sách `assigned_route`, `remaining_customers` và các thông tin về vị trí hiện tại của xe. Đồng thời, service truy vấn Redis để lấy danh sách cạnh bị tắc hoặc traffic state liên quan.

Khi tuyến hiện tại đi qua cạnh tắc nghẽn hoặc cạnh có vận tốc thấp, optimizer chạy Genetic Algorithm để tối ưu thứ tự giao khách. Sau đó, service xây dựng hoặc cập nhật `new_assigned_route` và ghi kết quả vào MongoDB. Cách thiết kế này giúp phân tối ưu hoạt động độc lập với stream processing: Spark chỉ chịu trách nhiệm tạo traffic state, còn optimizer quyết định cách dùng traffic state đó để cập nhật tuyến đường.

3.4.4. Dashboard backend

Dashboard backend là cầu nối giữa các service dữ liệu và frontend. Backend sử dụng Kafka consumer để nhận dữ liệu vị trí xe tải, dùng Redis để lấy traffic state và dùng MongoDB để đọc hoặc theo dõi route assignment. Dữ liệu sau khi được

tổng hợp sẽ được đẩy tới frontend qua Socket.IO.

Backend cũng sử dụng MongoDB Change Streams để theo dõi thay đổi trong collection `assigned_routes`. Khi route optimization service cập nhật tuyến đường mới, backend nhận sự kiện thay đổi và phát event `route_optimized` cho frontend. Cơ chế này giúp dashboard phản ánh kết quả tối ưu mà không cần frontend liên tục gọi API polling thủ công.

```
PS C:\dev\logistics-real-time> kubectl logs deployment/dashboard-backend --tail=60
PathFinder loaded: 9926 edges, 5560 nodes
Đang kết nối Kafka (kafka.default.svc.cluster.local:9092)...
SERVER RUNNING ON PORT 4080
Đã kết nối Redis
Đã kết nối MongoDB
MongoDB Change Streams đã sẵn sàng
Kafka: đã subscribe topic gps_stream
{"level":"INFO","timestamp":"2026-05-27T12:42:58.947Z","logger":"kafkajs","message":"[Consumer] Starting","groupId":"dashboard-group"}
{"level":"ERROR","timestamp":"2026-05-27T12:42:58.986Z","logger":"kafkajs","message":"[Connection] Response GroupCoordinator(key: 10, version: 2)","broker":
{"kafka.default.svc.cluster.local:9092","clientId":"dashboard-backend","error":"The group coordinator is not available","correlationId":2,"size":55}
{"level":"INFO","timestamp":"2026-05-27T12:43:05.496Z","logger":"kafkajs","message":"[ConsumerGroup] Consumer has joined the group","groupId":"dashboard-gro
up","memberId":"dashboard-backend-cf46aeaa-531b-448a-bc8e-779e75abf5dc","leaderId":"dashboard-backend-cf46aeaa-531b-448a-bc8e-779e75abf5dc","isLeader":true,
"memberAssignment":{"gps_stream":[0]}","groupProtocol":"RoundRobinAssigner","duration":6547}
Dashboard connected: Xplt2iR9MUyU4IyQAAAB
routes_snapshot: 100 routes, 100 with customers
Dashboard disconnected: Xplt2iR9MUyU4IyQAAAB
Dashboard connected: Mtyfm2fv3gCfsTRRAAAE
routes_snapshot: 100 routes, 100 with customers
```

Hình 3.2. Dashboard backend kết nối Kafka, Redis, MongoDB và sẵn sàng phát dữ liệu tới giao diện.

Hình 3.2. Cho thấy backend đã load graph, kết nối Redis, MongoDB, subscribe Kafka topic `gps_stream` và phát route snapshot cho dashboard. Đây là bằng chứng vận hành cho vai trò trung gian của backend giữa các tầng dữ liệu và giao diện.

3.4.5. Dashboard frontend

Dashboard frontend là lớp trực quan hóa của hệ thống. Frontend nhận các sự kiện gần thời gian thực từ backend, bao gồm batch vị trí phương tiện, traffic state theo cạnh đường và tuyến đường được tối ưu. Dữ liệu này được hiển thị trên bản đồ để người dùng quan sát trạng thái giao thông và tuyến đường của xe.

Frontend không trực tiếp kết nối Kafka, Redis hoặc MongoDB. Tất cả dữ liệu đều đi qua dashboard backend. Cách tách biệt này giúp giao diện đơn giản hơn, đồng thời tránh đưa thông tin kết nối hạ tầng trực tiếp ra phía trình duyệt.

3.5. Luồng xử lý end-to-end

Luồng xử lý end-to-end của hệ thống bắt đầu từ mô phỏng phương tiện và kết thúc ở dashboard gần thời gian thực. Mỗi bước trong pipeline biến đổi dữ liệu theo một mức ý nghĩa cao hơn: từ message GPS của phương tiện, sang trạng thái giao

thông theo từng cạnh đường, sang dữ liệu tuyến đường trong MongoDB và cuối cùng là thông tin trực quan hóa trên dashboard.

1. **Sinh dữ liệu:** Bot simulator cập nhật vị trí phương tiện theo từng tick và tạo message GPS dạng JSON. Message này gồm định danh phương tiện, tọa độ, vận tốc, thời điểm ghi nhận và `edge_id` hiện tại.
2. **Đưa vào Kafka:** GPS producer serialize message thành JSON bytes và gửi vào Kafka topic `gps_stream`.
3. **Xử lý streaming:** Spark Structured Streaming đọc message từ Kafka, parse JSON theo schema, lọc dữ liệu không hợp lệ và chuyển `timestamp` sang `event_time`.
4. **Nhóm theo cạnh đường:** Spark sử dụng `edge_id` có trong message để nhóm dữ liệu theo cạnh đường. Cách này giúp pipeline tránh phải thực hiện map-matching nặng trong mỗi micro-batch.
5. **Tổng hợp traffic:** Spark group dữ liệu theo `edge_id` và window thời gian để tính `avg_speed` và `vehicle_count`.
6. **Ghi traffic state:** Spark bổ sung chiều dài cạnh, thời gian di chuyển ước lượng và cờ `is_congested`, sau đó ghi traffic state vào Redis theo key `edge:<edge_id>`. Các cạnh có dấu hiệu tắc nghẽn được cập nhật vào set `blocked_edges`.
7. **Tối ưu tuyến:** Route optimization service kiểm tra route assignment trong MongoDB và trạng thái giao thông trong Redis để quyết định có cần cập nhật tuyến hay không. Khi cần, service chạy Genetic Algorithm để tối ưu thứ tự giao khách và cập nhật kết quả route vào MongoDB.
8. **Phát dữ liệu gần thời gian thực:** Dashboard backend nhận vị trí phương tiện từ Kafka, đọc traffic state từ Redis và đọc route từ MongoDB. Sau đó backend phát dữ liệu vị trí, traffic và route update cho frontend thông qua Socket.IO.

Luồng này thể hiện rõ vai trò của từng công nghệ trong hệ thống. Kafka giúp tiếp nhận và phân phối dòng sự kiện GPS, Spark tổng hợp message GPS

thành traffic state theo cạnh đường, Redis cung cấp trạng thái giao thông ngắn hạn, MongoDB lưu dữ liệu tuyến đường và kết quả tối ưu, còn Socket.IO giúp dashboard cập nhật dữ liệu gần thời gian thực.

3.6. Giao tiếp giữa các service

Các service trong hệ thống giao tiếp với nhau thông qua ba cơ chế chính: message streaming, truy vấn state và sự kiện gần thời gian thực. Việc sử dụng nhiều cơ chế giao tiếp khác nhau phản ánh đặc điểm của từng loại dữ liệu trong hệ thống.

Cơ chế	Thành phần sử dụng	Mục đích
Kafka topic	Bot simulator; Spark; dashboard backend	Truyền dữ liệu GPS dạng sự kiện giữa producer và consumer.
Redis key-value/set	Spark; route optimization; dashboard backend	Lưu và truy vấn traffic state ngắn hạn theo cạnh đường.
MongoDB collection	Route optimization; dashboard backend	Lưu route assignment và kết quả tối ưu tuyến đường.
MongoDB Change Streams	Dashboard backend	Theo dõi thay đổi route assignment để phát sự kiện gần thời gian thực.
Socket.IO	Dashboard backend; frontend	Đẩy dữ liệu vị trí, traffic và tuyến đường tới giao diện người dùng.
Kubernetes Service DNS	Tất cả microservice	Cho phép các pod kết nối tới nhau bằng service name nội bộ.

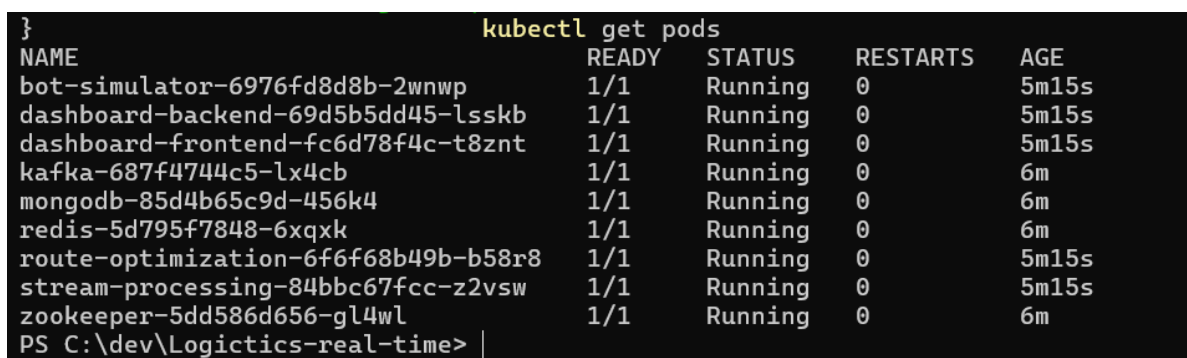
Bảng 3.3. Các cơ chế giao tiếp chính giữa các service

Kafka phù hợp với dữ liệu sự kiện có tần suất cao như GPS. Redis phù hợp với trạng thái ngắn hạn cần truy vấn nhanh. MongoDB phù hợp với dữ liệu có cấu trúc và cần theo dõi thay đổi. Socket.IO phù hợp với luồng dữ liệu đẩy từ backend ra frontend. Sự kết hợp này giúp hệ thống vừa xử lý được dữ liệu streaming, vừa

hiển thị được kết quả gần thời gian thực.

3.7. Triển khai bằng Kubernetes

Hệ thống được triển khai trên Kubernetes bằng hai nhóm YAML chính: 01-infrastructure.yaml và 02-microservices.yaml. File đầu tiên triển khai các service hạ tầng như Zookeeper, Kafka, Redis và MongoDB. File thứ hai triển khai các microservice nghiệp vụ như bot simulator, stream processing, route optimization, dashboard backend và dashboard frontend.



NAME	READY	STATUS	RESTARTS	AGE
bot-simulator-6976fd8d8b-2wnwp	1/1	Running	0	5m15s
dashboard-backend-69d5b5dd45-lsskb	1/1	Running	0	5m15s
dashboard-frontend-fc6d78f4c-t8znt	1/1	Running	0	5m15s
kafka-687f4744c5-lx4cb	1/1	Running	0	6m
mongodb-85d4b65c9d-456k4	1/1	Running	0	6m
redis-5d795f7848-6xqyk	1/1	Running	0	6m
route-optimization-6f6f68b49b-b58r8	1/1	Running	0	5m15s
stream-processing-84bbc67fcc-z2vsw	1/1	Running	0	5m15s
zookeeper-5dd586d656-gl4wl	1/1	Running	0	6m

Hình 3.3. Trạng thái các thành phần hệ thống khi triển khai trên Kubernetes

Hình 3.3. Minh họa trạng thái các pod trong lần chạy thử nghiệm cuối. Tất cả thành phần chính đều ở trạng thái Running, cho phép tiếp tục kiểm chứng luồng dữ liệu end-to-end.

Trong môi trường Kubernetes, mỗi thành phần được triển khai bằng một Deployment với một replica. Các service trao đổi với nhau thông qua địa chỉ DNS nội bộ, ví dụ kafka.default.svc.cluster.local:9092, redis.default.svc.cluster.local:6379 và mongodb.default.svc.cluster.local:27017. Dashboard backend và frontend được expose qua NodePort để người dùng có thể truy cập từ bên ngoài cluster.

Service	Port	Vai trò trong triển khai
zookeeper	2181	Điều phối cho Kafka trong cấu hình hiện tại.
kafka	9092	Message broker cho topic <code>gps_stream</code> .
redis	6379	Lưu traffic state và tập <code>blocked_edges</code> .
mongodb	27017	Lưu dữ liệu route assignment và kết quả tối ưu.
dashboard-backend	4000	Backend gần thời gian thực, nhận dữ liệu từ Kafka, Redis và MongoDB, sau đó phát sự kiện Socket.IO cho frontend.
dashboard-frontend	5173	Giao diện dashboard trên trình duyệt.

Bảng 3.4. Các service và cổng chính trong Kubernetes

Cách triển khai này phù hợp với môi trường bài tập lớn và kiểm thử cục bộ vì dễ khởi động, dễ quan sát bằng lệnh `kubectl get pods`, `kubectl logs` và dễ khởi động lại từng service khi cần. Tuy nhiên, vì mỗi service hiện chỉ chạy một replica, hệ thống chưa hướng tới high availability như một môi trường production thực sự.

3.8. Đặc điểm và giới hạn kiến trúc

Kiến trúc hiện tại có ưu điểm là rõ ràng, dễ mở rộng theo từng tầng và phù hợp với pipeline dữ liệu lớn. Kafka giúp giảm phụ thuộc giữa simulator và consumer; Spark đảm nhiệm xử lý streaming; Redis giúp truy vấn traffic state nhanh; MongoDB lưu dữ liệu tuyến đường; dashboard backend gom dữ liệu từ nhiều nguồn và đẩy ra frontend.

Tuy vậy, hệ thống vẫn có một số giới hạn do phạm vi của bài tập lớn. Thứ nhất, các service hạ tầng như Kafka, Redis và MongoDB chỉ chạy với một replica, nên chưa có khả năng chịu lỗi cao. Thứ hai, dữ liệu GPS là dữ liệu mô phỏng, chưa tích hợp nguồn dữ liệu giao thông thật. Thứ ba, route optimization vẫn được thiết

kế như một worker quét định kỳ, chưa phải một hệ thống tối ưu phân tán quy mô lớn. Những giới hạn này phù hợp với phạm vi bài tập lớn và môi trường triển khai cục bộ, đồng thời cần được xem xét thêm nếu hệ thống được phát triển ở quy mô sản xuất.

Tổng thể, kiến trúc đã đáp ứng được mục tiêu chính của dự án: xây dựng một pipeline logistics gần thời gian thực end-to-end, từ sinh dữ liệu GPS, xử lý streaming, lưu trạng thái giao thông, tối ưu tuyến đường đến hiển thị trên dashboard.

CHƯƠNG 4. XỬ LÝ DỮ LIỆU STREAMING BẰNG SPARK

4.1. Vai trò của tầng stream processing

Trong hệ thống logistics gần thời gian thực, tầng stream processing là thành phần chuyển đổi luồng GPS thô thành trạng thái giao thông có thể sử dụng được. Nếu Kafka đóng vai trò tiếp nhận và phân phối sự kiện, thì Spark Structured Streaming đảm nhận nhiệm vụ đọc dữ liệu liên tục từ Kafka, chuẩn hóa bản ghi GPS, ánh xạ tọa độ vào cạnh đường, tổng hợp vận tốc theo cửa sổ thời gian và ghi kết quả traffic state vào Redis.

Điểm quan trọng của tầng này là dữ liệu không được xử lý theo từng file tĩnh, mà được xử lý như một dòng sự kiện liên tục. Mỗi phương tiện trong simulator sinh ra bản ghi GPS theo chu kỳ; các bản ghi này được đưa vào Kafka topic `gps_stream`; Spark đọc topic này bằng cơ chế streaming và xử lý theo micro-batch. Nhờ đó, trạng thái giao thông trên từng cạnh đường được cập nhật liên tục và có thể phục vụ dashboard cũng như route optimization service.

Trong triển khai hiện tại, module xử lý streaming nằm tại file `stream_processing/spark_kafka_consumer.py`. Module này kết hợp ba nhóm logic chính: đọc và parse dữ liệu Kafka, map-matching GPS sang `edge_id`, sau đó aggregate theo window để ghi traffic state vào Redis. Ngoài Redis, hệ thống còn ghi dữ liệu GPS đã được map-matching ra Parquet như một hướng lưu trữ dài hạn phục vụ kiểm tra hoặc phân tích sau này.

4.2. Cấu hình Spark Structured Streaming

Spark Structured Streaming được khởi tạo bằng `SparkSession` với tên ứng dụng `LogisticsProcessing`. Do hệ thống chạy trong container cục bộ và Kubernetes local, cấu hình bộ nhớ driver và executor được giới hạn ở mức vừa phải để tránh gây quá tải tài nguyên máy cá nhân.

Tham số	Giá trị hiện tại	Ý nghĩa
KAFKA_TOPIC	gps_stream	Topic chứa dữ liệu GPS từ bot simulator.
KAFKA_BROKER	Service Kafka trong Kubernetes	Địa chỉ Kafka broker, được truyền qua biến môi trường khi triển khai.
WATERMARK_DELAY	10 seconds	Cho phép dữ liệu đến muộn trong khoảng 10 giây.
WINDOW_DURATION	10 seconds	Độ dài cửa sổ tổng hợp traffic.
SLIDE_DURATION	5 seconds	Chu kỳ trượt của cửa sổ xử lý.
CONGESTION_THRESHOLD_KMH	10.0	Ngưỡng vận tốc trung bình để Spark đánh giá một cạnh đường là tắc nghẽn.
maxOffsetsPerTrigger	20000	Giới hạn số message Kafka tối đa được đọc trong mỗi trigger.
processingTime	5 seconds	Chu kỳ kích hoạt micro-batch ghi kết quả sang Redis.

Bảng 4.1. Các tham số chính của Spark Structured Streaming

Trong Docker image của stream processing, dự án sử dụng Java runtime, Python, PySpark 3.5.0, Redis client, Confluent Kafka client và Shapely. Khi chạy Spark, package `spark-sql-kafka-0-10_2.13:3.5.0` được khai báo để Spark có thể đọc dữ liệu trực tiếp từ Kafka. Việc cố định version PySpark và Kafka connector giúp giảm rủi ro lệch phiên bản trong quá trình build và triển khai.

4.3. Đọc dữ liệu GPS từ Kafka

Nguồn dữ liệu streaming của Spark là Kafka topic `gps_stream`. Spark đọc topic này bằng `readStream`, sử dụng định dạng `kafka`. Mỗi message Kafka ban đầu có phần value ở dạng bytes, do đó module cần cast value sang string trước khi parse JSON.

Schema mà Spark sử dụng trong pipeline gồm các trường cột lỗi của

bản ghi GPS: `entity_id`, `entity_type`, `latitude`, `longitude`, `speed` và `timestamp`. Mặc dù simulator có thể gửi thêm `edge_id`, `route_index` và `route_total` đối với xe tải, Spark không phụ thuộc trực tiếp vào các trường này trong bước tổng hợp traffic. Thay vào đó, Spark thực hiện map-matching lại từ tọa độ GPS để xác định cạnh đường tương ứng.

Trường Spark parse	Kiểu dữ liệu	Vai trò trong pipeline
<code>entity_id</code>	String	Định danh phương tiện, dùng khi đếm số lượng xe trong window.
<code>entity_type</code>	String	Lọc các bản ghi hợp lệ, gồm Truck và Bot.
<code>latitude</code>	Double	Tọa độ đầu vào cho map-matching.
<code>longitude</code>	Double	Tọa độ đầu vào cho map-matching.
<code>speed</code>	Double	Vận tốc hiện tại, dùng để tính <code>avg_speed</code> .
<code>timestamp</code>	Long	Epoch milliseconds, được chuyển thành <code>event_time</code> .

Bảng 4.2. Schema GPS được Spark parse từ Kafka

Sau khi parse JSON, Spark lọc bỏ các bản ghi không thuộc hai loại Truck hoặc Bot, đồng thời bỏ các bản ghi không có tọa độ hợp lệ. Trường `timestamp` được chuyển sang `event_time` để sử dụng cho watermark và window aggregation. Đây là bước cần thiết vì Spark Structured Streaming xử lý dữ liệu theo event time, không chỉ theo thời điểm message được đọc từ Kafka.

hợp traffic, hệ thống cần thực hiện map-matching để ánh xạ mỗi cặp `latitude`, `longitude` sang một `edge_id`.

Trong dự án, map-matching được hiện thực trong module `map_matching.py`. Khi khởi tạo, module đọc file `edges_schema.json` và lưu danh sách cạnh đường vào bộ nhớ. Với mỗi điểm GPS, thuật toán duyệt qua các cạnh đường, tính khoảng cách từ điểm GPS tới đoạn thẳng nối `start_node` và `end_node`, sau đó chọn cạnh có khoảng cách nhỏ nhất. Nếu dữ liệu cạnh chưa được load hoặc có lỗi xử lý, hệ thống trả về UNKNOWN để tránh làm crash pipeline.

Thành phần	Vai trò
MapMatcher	Load <code>edges_schema.json</code> , lưu <code>edge_map</code> và danh sách cạnh đường.
<code>snap_to_edge</code>	Tìm <code>edge_id</code> gần nhất với tọa độ GPS.
<code>get_edge_length</code>	Lấy chiều dài cạnh, dùng để tính thời gian di chuyển ước lượng.
<code>get_edge_max_speed</code>	Lấy tốc độ tham chiếu của cạnh, ghi vào <code>traffic state</code> .
UNKNOWN	Giá trị fallback khi không tìm được cạnh hợp lệ.

Bảng 4.3. Các thành phần chính trong bước map-matching

Map-matching được gọi trong Spark thông qua UDF `match_to_edge_udf`. Sau bước này, mỗi bản ghi GPS hợp lệ được bổ sung thêm trường `edge_id`. Đây là field trung tâm của toàn bộ tầng stream processing, vì các thống kê traffic đều được nhóm theo `edge_id`.

4.5. Watermark, window và aggregate

Sau khi có `edge_id`, Spark tổng hợp dữ liệu theo cửa sổ thời gian để tạo ra traffic state cho từng cạnh đường. Pipeline sử dụng event-time window với watermark nhằm xử lý dữ liệu đến muộn trong phạm vi cho phép.

Cụ thể, Spark áp dụng watermark 10 seconds trên trường `event_time`, sau đó group dữ liệu theo cặp `window(event_time)` và `edge_id`. Với mỗi nhóm, hệ thống tính vận tốc trung bình bằng `avg(speed)` và số lượng phương

tiện bằng `count(entity_id)`. Kết quả được đặt tên thành `avg_speed` và `vehicle_count`.

Chỉ số	Cách tính	Ý nghĩa
<code>avg_speed</code>	<code>avg(speed)</code> theo <code>edge_id</code> và <code>window</code>	Phản ánh vận tốc trung bình trên cạnh đường trong cửa sổ thời gian gần nhất.
<code>vehicle_count</code>	<code>count(entity_id)</code> theo <code>edge_id</code> và <code>window</code>	Ước lượng số lượng phương tiện xuất hiện trên cạnh trong <code>window</code> xử lý.
<code>edge_length_m</code>	Lookup từ <code>edges_schema.json</code>	Chiều dài cạnh đường, dùng để ước lượng thời gian di chuyển.
<code>estimated_travel_time</code>	$(\text{edge_length_m} / 1000) / \text{avg_speed} * 3600$	Thời gian di chuyển ước lượng qua cạnh, đơn vị giây.
<code>is_congested</code>	<code>avg_speed < 10.0</code>	Đánh dấu nội bộ rằng cạnh đường đang có vận tốc trung bình thấp hơn ngưỡng tắc nghẽn.

Bảng 4.4. Các chỉ số traffic được tạo từ Spark aggregation

Sau `aggregate`, Spark bổ sung chiều dài cạnh bằng UDF `get_edge_length_udf`. DataFrame kết quả được repartition thành 4 partition trước khi ghi Redis. Việc repartition này giúp phân tán việc ghi dữ liệu theo nhiều partition, giảm nguy cơ một partition xử lý quá nhiều record trong cùng batch.

4.6. Ghi traffic state vào Redis

Sau khi `aggregate` và `enrich` dữ liệu, Spark ghi kết quả traffic state vào Redis thông qua `foreachBatch`. Với mỗi micro-batch, hệ thống duyệt từng partition bằng `foreachPartition`, tạo Redis writer riêng cho partition và ghi nhiều record bằng pipeline. Cách làm này phù hợp hơn việc ghi từng record một, vì giảm số lần round-trip tới Redis.

Mỗi record traffic được ghi vào Redis dưới dạng JSON string với key có dạng `edge:<edge_id>`. TTL hiện tại là 120 giây. Điều này phù hợp với bản chất của

traffic state: dữ liệu giao thông chỉ có giá trị trong khoảng thời gian ngắn và không nên tồn tại quá lâu nếu không được cập nhật.

Cơ chế Redis	Ý nghĩa trong hệ thống
edge:<edge_id>	Key lưu traffic state mới nhất của một cạnh đường.
JSON string	Format lưu payload traffic gồm tốc độ, số lượng xe, thời gian ước lượng và cờ tắc nghẽn.
TTL 120 giây	Tự động loại bỏ traffic state cũ để tránh sử dụng dữ liệu lỗi thời.
blocked_edges	Set lưu các cạnh đang có is_congested = true.
Redis pipeline	Ghi nhiều key trong một batch để giảm overhead.

Bảng 4.5. Cơ chế ghi traffic state từ Spark vào Redis

Payload ghi vào Redis gồm các trường chính sau:

- edge_id: mã cạnh đường tương ứng với traffic state.
- avg_speed: vận tốc trung bình trên cạnh trong window xử lý, đơn vị km/h.
- vehicle_count: số lượng phương tiện được ghi nhận trên cạnh trong window.
- distance: chiều dài cạnh đường, đơn vị mét.
- estimated_travel_time: thời gian di chuyển ước lượng qua cạnh, đơn vị giây.
- is_congested: trạng thái nội bộ cho biết cạnh có vận tốc trung bình thấp hơn ngưỡng tắc nghẽn hay không.
- updated_at: thời điểm kết thúc window dùng để cập nhật traffic state.

Trong đó, is_congested được tính bằng cách so sánh avg_speed với CONGESTION_THRESHOLD_KMH. Nếu một cạnh bị đánh dấu tắc nghẽn, Redis writer thêm edge_id vào set blocked_edges; nếu không, cạnh đó bị loại khỏi set này. Cấu trúc này giúp dashboard đọc nhanh trạng thái giao thông theo từng cạnh, đồng thời giúp route optimizer kiểm tra các cạnh bất lợi khi cần cập nhật tuyến đường.

4.7. Ghi dữ liệu GPS ra Parquet

Bên cạnh việc ghi traffic state ngắn hạn vào Redis, pipeline còn ghi dữ liệu GPS đã được map-matching ra Parquet. Đây là hướng lưu trữ phục vụ phân tích sau chạy, kiểm tra dữ liệu và tái hiện lại một phần luồng GPS nếu cần.

Parquet sink sử dụng output mode `append`, ghi các trường `entity_id`, `entity_type`, `latitude`, `longitude`, `speed`, `timestamp`, `event_time` và `edge_id`. Chu kỳ trigger của Parquet sink là 30 giây, dài hơn chu kỳ ghi Redis vì dữ liệu lưu trữ dài hạn không cần cập nhật nhanh như traffic state gần thời gian thực.

Trong cấu hình thử nghiệm hiện tại, đường dẫn mặc định của Parquet là `/tmp/gps_parquet`. Đây là lựa chọn phù hợp cho thử nghiệm cục bộ, nhưng nếu triển khai production cần thay bằng persistent volume hoặc object storage để tránh mất dữ liệu khi pod bị xóa hoặc container khởi động lại.

4.8. Triển khai stream processing trong Kubernetes

Trong Kubernetes, stream processing được triển khai dưới dạng một deployment riêng tên `stream-processing`. Container sử dụng image `logictics-stream:v5`, nhận các biến môi trường để kết nối tới Kafka, Redis và file dữ liệu bản đồ.

Biến môi trường	Giá trị trong Kubernetes
KAFKA_BROKER	<code>kafka.default.svc.cluster.local:9092</code>
REDIS_HOST	<code>redis.default.svc.cluster.local</code>
REDIS_PORT	<code>6379</code>
EDGES_JSON	<code>/app/data/edges_schema.json</code>

Bảng 4.6. Biến môi trường của deployment `stream-processing`

Việc tách stream processing thành service độc lập giúp hệ thống dễ debug và dễ scale hơn. Khi pipeline gặp lỗi Kafka, Redis hoặc lỗi schema, nhóm có thể kiểm tra riêng log của deployment `stream-processing` mà không ảnh hưởng trực tiếp tới bot simulator, route optimization hay dashboard backend.

```

PS C:\dev\Logistics-real-time> kubectl logs deployment/stream-processing --tail=80
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379
INFO:StreamProcessor:[Batch 65] Ghi 1491 edges → Redis.
INFO:StreamProcessor:[Batch 65] Ghi 1436 edges → Redis.
INFO:StreamProcessor:[Batch 65] Ghi 1432 edges → Redis.
INFO:StreamProcessor:[Batch 65] Ghi 1473 edges → Redis.
2026-05-27 12:48:21,428 [INFO] StreamProcessor - [Batch 65] [✓] Done.
2026-05-27 12:48:25,118 [INFO] py4j.clientserver - Received command c on object id p0
2026-05-27 12:48:25,775 [INFO] StreamProcessor - [Batch 66] Xử lý 5132 records...
26/05/27 12:48:25 WARN HDFSBackedStateStoreProvider: The state for version 66 doesn't exist in loadedMaps. Reading snapshot file and delta files if needed..
Note that this is normal for the first batch of starting query.
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379
INFO:StreamProcessor:[Batch 66] Ghi 1302 edges → Redis.
INFO:StreamProcessor:[Batch 66] Ghi 1277 edges → Redis.
INFO:StreamProcessor:[Batch 66] Ghi 1251 edges → Redis.
INFO:StreamProcessor:[Batch 66] Ghi 1302 edges → Redis.
2026-05-27 12:48:26,152 [INFO] StreamProcessor - [Batch 66] [✓] Done.
2026-05-27 12:48:30,144 [INFO] py4j.clientserver - Received command c on object id p0
2026-05-27 12:48:31,184 [INFO] StreamProcessor - [Batch 67] Xử lý 5818 records...
26/05/27 12:48:31 WARN HDFSBackedStateStoreProvider: The state for version 67 doesn't exist in loadedMaps. Reading snapshot file and delta files if needed..
Note that this is normal for the first batch of starting query.
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379
INFO:StreamProcessor:[Batch 67] Ghi 1490 edges → Redis.
INFO:StreamProcessor:[Batch 67] Ghi 1468 edges → Redis.
INFO:StreamProcessor:[Batch 67] Ghi 1410 edges → Redis.
INFO:StreamProcessor:[Batch 67] Ghi 1450 edges → Redis.
2026-05-27 12:48:31,443 [INFO] StreamProcessor - [Batch 67] [✓] Done.
2026-05-27 12:48:35,122 [INFO] py4j.clientserver - Received command c on object id p0
2026-05-27 12:48:35,745 [INFO] StreamProcessor - [Batch 68] Xử lý 5154 records...
26/05/27 12:48:35 WARN HDFSBackedStateStoreProvider: The state for version 68 doesn't exist in loadedMaps. Reading snapshot file and delta files if needed..
Note that this is normal for the first batch of starting query.
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379
INFO:RedisWriter: [✓] Redis Connected: redis.default.svc.cluster.local:6379

```

Hình 4.3. Spark Structured Streaming xử lý micro-batch và ghi trạng thái giao thông vào Redis

Hình 4.3 cho thấy Spark xử lý các micro-batch liên tiếp và ghi kết quả tổng hợp theo cạnh đường vào Redis. Đây là phần thể hiện rõ nhất vai trò của Spark Structured Streaming trong dự án Big Data.

4.9. Nhận xét và giới hạn

Tầng Spark Structured Streaming đã đáp ứng được mục tiêu chính của dự án: biến dòng GPS thô thành traffic state theo từng cạnh đường trong cơ chế gần thời gian thực. Thiết kế sử dụng Kafka làm source, Spark làm processing engine, Redis làm state store ngắn hạn và Parquet làm hướng lưu trữ dài hạn phản ánh đúng tinh thần của một pipeline dữ liệu lớn.

Tuy vậy, pipeline hiện tại vẫn có một số giới hạn xuất phát từ phạm vi bài tập lớn và môi trường triển khai cục bộ. Thứ nhất, dữ liệu mô phỏng hiện đã gửi kèm `edge_id`, giúp Spark tổng hợp traffic state theo cạnh đường mà không phải thực hiện map-matching nặng trong mỗi micro-batch. Cách làm này phù hợp với phạm vi thử nghiệm, nhưng nếu tích hợp GPS thật thì hệ thống cần bổ sung bước map-matching chặt chẽ hơn. Thứ hai, checkpoint và dữ liệu lịch sử mới dừng ở mức phục vụ thử nghiệm, chưa có persistent volume đầy đủ cho môi trường triển khai lâu dài. Thứ ba, ngưỡng tắc nghẽn 10 km/h là rule-based, dễ hiểu và dễ kiểm

thử, nhưng chưa phải mô hình traffic phức tạp.

Dù còn giới hạn, tầng stream processing đã tạo được liên kết cốt lõi giữa dữ liệu gần thời gian thực và các module phía sau. Redis traffic state là đầu vào quan trọng cho dashboard hiển thị mật độ giao thông và cho route optimization phát hiện các cạnh cần tránh. Vì vậy, Spark Structured Streaming đóng vai trò trung tâm trong việc biến simulator thành một hệ thống logistics có khả năng phản ứng với thay đổi giao thông.

CHƯƠNG 5. TỐI ƯU HÓA TUYẾN ĐƯỜNG

5.1. Vai trò của route optimization

Trong hệ thống logistics gần thời gian thực, route optimization là tầng sử dụng trạng thái giao thông hiện tại để hỗ trợ cập nhật tuyến đường cho xe giao hàng. Nếu tầng Spark tạo ra traffic state theo từng cạnh đường, thì tầng tối ưu tuyến đường sử dụng traffic state này cùng với dữ liệu tuyến trong MongoDB để quyết định có cần tối ưu lại hay không. Khi cần tối ưu, hệ thống dùng Genetic Algorithm để tìm thứ tự giao khách phù hợp hơn, sau đó dùng Dijkstra/graph route building để sinh danh sách cạnh đường mới và lưu kết quả trở lại MongoDB.

Module này không xử lý toàn bộ luồng GPS trực tiếp. Thay vào đó, nó hoạt động như một worker chạy nền, định kỳ đọc các bản ghi route assignment trong collection `assigned_routes`. Với mỗi xe, worker chuẩn hóa dữ liệu đầu vào, lấy thông tin tắc nghẽn từ Redis, kiểm tra route hiện tại và chỉ chạy thuật toán khi route có dấu hiệu bị ảnh hưởng bởi cạnh tắc hoặc cạnh có vận tốc thấp.

Thiết kế này giúp tách biệt rõ ba nhiệm vụ: Spark chịu trách nhiệm phát hiện trạng thái giao thông, Redis lưu trạng thái ngắn hạn, còn route optimization tập trung vào quyết định tối ưu tuyến giao hàng. Kết quả tối ưu được ghi vào MongoDB để dashboard backend có thể theo dõi thay đổi và phát sự kiện gần thời gian thực tới frontend.

5.2. Dữ liệu đầu vào của optimizer

Dữ liệu đầu vào của optimizer đến từ hai nguồn chính. Nguồn thứ nhất là MongoDB, lưu thông tin route assignment của từng xe. Nguồn thứ hai là Redis, lưu traffic state ngắn hạn và tập các cạnh bị đánh dấu tắc nghẽn. Trước khi chạy thuật toán, hệ thống cần đưa hai nguồn dữ liệu này về một cấu trúc thống nhất.

Trong code, bước chuẩn hóa này được hiện thực trong module `optimizer_input_adapter`. Module này hỗ trợ cả hai cách đặt tên `entity_id` và `vehicle_id`, sau đó tạo ra một object đầu vào thống nhất cho thuật toán tối

ưu.

Nhóm dữ liệu	Vai trò
Thông tin xe	Xác định <code>vehicle_id</code> , <code>entity_id</code> , <code>entity_type</code> và cạnh hiện tại của xe.
Tuyến hiện tại	Danh sách <code>edge_id</code> trong <code>assigned_route</code> , dùng để kiểm tra route có đi qua cạnh tắc hay không.
Khách hàng còn lại	Danh sách điểm giao hàng chưa hoàn thành, gồm <code>cust_id</code> , <code>latitude</code> và <code>longitude</code> .
Cạnh bị tắc	Lấy từ Redis set <code>blocked_edges</code> .
Traffic snapshot	Trạng thái traffic của các cạnh quan trọng, gồm cạnh hiện tại, route hiện tại và các cạnh bị tắc.

Bảng 5.1. Các nhóm dữ liệu đầu vào của route optimization

Sau khi chuẩn hóa, đầu vào của thuật toán có các trường như `vehicle_id`, `current_edge_id`, `assigned_route`, `remaining_customers`, `blocked_edges` và `traffic_snapshot`. Đây là lớp trung gian quan trọng vì thuật toán Genetic Algorithm không cần biết trực tiếp cách đọc MongoDB hoặc Redis.

5.3. Điều kiện kích hoạt tối ưu lại tuyến

Không phải mọi route đều cần được tối ưu lại ở mọi thời điểm. Nếu worker luôn chạy GA cho toàn bộ xe sau mỗi chu kỳ, hệ thống sẽ lãng phí tài nguyên và khó mở rộng. Vì vậy, route optimization sử dụng cơ chế trigger để quyết định khi nào cần reroute.

Về mặt thiết kế, hệ thống kích hoạt tối ưu lại tuyến khi một trong các điều kiện sau xảy ra:

1. Route hiện tại có giao với tập `blocked_edges` trong Redis.
2. Một cạnh trong `assigned_route` có `is_congested = true`.
3. Vận tốc trung bình của một cạnh trong route nhỏ hơn hoặc bằng ngưỡng `min_avg_speed`.

4. Xe chưa có `new_assigned_route`, khi đó worker force tối ưu một lần để bootstrap dữ liệu cho dashboard.

Cách thiết kế này giúp optimizer chỉ tập trung vào các xe thực sự cần xử lý. Trong trường hợp route hiện tại không bị ảnh hưởng bởi giao thông xấu, worker trả về trạng thái `skipped`. Điều này cũng làm cho log hệ thống rõ ràng hơn: route nào được tối ưu, route nào bị bỏ qua và route nào gặp lỗi.

5.4. Genetic Algorithm cho thứ tự giao khách

Phần lõi của module tối ưu là Genetic Algorithm. Trong bài toán này, mỗi cá thể trong quần thể biểu diễn một thứ tự giao khách. Ví dụ, với ba khách hàng còn lại, một cá thể có thể là thứ tự `[Cust_003, Cust_001, Cust_002]`. Mục tiêu của thuật toán là tìm thứ tự có chi phí di chuyển ước lượng nhỏ nhất, đồng thời tránh hoặc phạt các route liên quan đến cạnh tắc nghẽn.

Các tham số mặc định của GA trong code gồm population size bằng 30, số generation bằng 60, mutation rate bằng 0.15 và elite size bằng 2. Các tham số này được chọn ở mức vừa phải để phù hợp với môi trường kiểm thử cục bộ, tránh thời gian tính toán quá lớn nhưng vẫn đủ ổn định để kiểm chứng vòng lặp tối ưu.

Tham số	Giá trị	Ý nghĩa
population_size	30	Số cá thể trong mỗi thế hệ.
generations	60	Số thế hệ GA chạy trước khi trả kết quả.
mutation_rate	0.15	Xác suất hoán đổi vị trí hai khách hàng trong một cá thể.
elite_size	2	Số cá thể tốt nhất được giữ lại trực tiếp sang thế hệ sau.
min_avg_speed	10.0	Ngưỡng vận tốc dùng để phát hiện cạnh có tốc độ thấp.

Bảng 5.2. Các tham số chính của Genetic Algorithm

Quy trình GA gồm các bước chính: khởi tạo quần thể bằng các hoán vị ngẫu nhiên của danh sách khách hàng, đánh giá fitness của từng cá thể, chọn nhóm cha

mẹ tốt hơn, lai ghép bằng order crossover, đột biến bằng cách hoán đổi hai khách hàng và lặp lại quá trình qua nhiều thế hệ.

5.5. Hàm fitness và chi phí di chuyển

Hàm fitness là thành phần quyết định chất lượng của mỗi cá thể. Trong triển khai hiện tại, fitness càng nhỏ thì cá thể càng tốt. Chi phí chính được ước lượng dựa trên khoảng cách giữa các khách hàng và tốc độ trung bình có được từ traffic snapshot.

Vì dữ liệu khách hàng được biểu diễn bằng tọa độ, thuật toán sử dụng khoảng cách Haversine để ước lượng khoảng cách giữa hai điểm. Sau đó, chi phí di chuyển được tính bằng khoảng cách chia cho tốc độ trung bình khả dụng. Nếu traffic snapshot không có dữ liệu tốc độ phù hợp, hệ thống dùng tốc độ mặc định để tránh làm thuật toán bị gián đoạn. Cách làm này giúp module dễ kiểm thử và phù hợp với mục tiêu tích hợp của bài tập lớn.

Ngoài chi phí khoảng cách, fitness còn cộng thêm penalty khi route hiện tại chứa cạnh bị tắc hoặc khi traffic snapshot cho thấy cạnh có tốc độ thấp. Điều này giúp GA đưa tín hiệu giao thông vào quá trình đánh giá mà không làm thuật toán phụ thuộc quá chặt vào một mô hình traffic phức tạp. Các penalty lớn cũng giúp tách biệt rõ route không phù hợp với route bình thường trong quá trình selection.

Thành phần fitness	Mô tả
Chi phí từ xe tới khách đầu tiên	Ước lượng từ cạnh hiện tại nếu traffic snapshot có dữ liệu, nếu không dùng giá trị mặc định an toàn.
Chi phí giữa các khách hàng	Tính bằng khoảng cách Haversine chia cho tốc độ trung bình khả dụng.
Penalty cạnh bị chặn	Cộng penalty rất lớn nếu route hiện tại giao với <code>blocked_edges</code> .
Penalty cạnh tắc nghẽn	Cộng penalty nếu cạnh trong route có <code>is_congested</code> hoặc <code>avg_speed</code> thấp.
Penalty cá thể không hợp lệ	Cộng penalty rất lớn nếu cá thể thiếu hoặc trùng khách hàng.

Bảng 5.3. Các thành phần của hàm fitness

Cách thiết kế fitness này phù hợp với trọng tâm của môn Big Data. Thuật toán chưa nhằm thay thế các mô hình tối ưu giao thông chuyên sâu, nhưng đã thể hiện được cơ chế đưa tín hiệu traffic state vào quyết định cập nhật tuyến, đồng thời giữ cho module đủ rõ ràng để debug trong một pipeline nhiều thành phần.

5.6. Selection, crossover và mutation

Sau khi đánh giá toàn bộ quần thể, thuật toán chọn các cá thể tốt hơn làm cha mẹ cho thế hệ tiếp theo. Cơ chế selection hiện tại sắp xếp quần thể theo fitness tăng dần và giữ lại nhóm tốt nhất. Ngoài ra, một số elite cá thể được đưa thẳng sang thế hệ sau để tránh mất nghiệm tốt đã tìm được.

Crossover sử dụng Order Crossover. Phương pháp này phù hợp với bài toán hoán vị vì nó giữ được một đoạn thứ tự từ cha mẹ thứ nhất, sau đó điền các khách hàng còn lại theo thứ tự xuất hiện trong cha mẹ thứ hai. Nhờ vậy, cá thể con không bị trùng khách và không bị thiếu khách, miễn là hai cá thể cha mẹ đều hợp lệ.

Mutation được hiện thực bằng cách hoán đổi vị trí hai khách hàng trong một cá thể với xác suất `mutation_rate`. Đây là thao tác đơn giản nhưng hiệu quả với bài toán tối ưu thứ tự, vì nó tạo ra biến thể mới mà vẫn giữ cá thể là một hoán vị hợp lệ của danh sách khách hàng.

Toán tử GA	Cách hiện thực trong dự án
Initialization	Random shuffle danh sách khách hàng để tạo quần thể ban đầu.
Fitness evaluation	Tính chi phí của từng thứ tự giao khách và cộng penalty giao thông.
Selection	Giữ nhóm cá thể có fitness thấp nhất làm cha mẹ.
Elitism	Giữ trực tiếp một số cá thể tốt nhất sang thế hệ sau.
Order crossover	Lấy một đoạn thứ tự từ cha mẹ thứ nhất và điền phần còn lại theo cha mẹ thứ hai.
Mutation	Hoán đổi vị trí hai khách hàng trong cá thể.

Bảng 5.4. Các toán tử chính trong Genetic Algorithm

5.7. Sinh tuyến cạnh mới bằng Dijkstra trên graph

Kết quả trực tiếp của GA là thứ tự giao khách đã tối ưu, nhưng dashboard cần một tuyến đường ở mức danh sách cạnh để vẽ polyline. Vì vậy, sau khi có thứ tự khách hàng, hệ thống dùng Dijkstra trên graph đường để sinh `new_assigned_route`. Đây là bước chuyển kết quả thuật toán từ mức “thứ tự điểm giao” sang mức dữ liệu vận hành mà dashboard và simulator có thể sử dụng.

Trong route optimization, graph được load từ file `edges_schema.json`. Mỗi cạnh có `start_node`, `end_node`, `length_meters` và `max_speed_kmh`. `GraphNetwork` xây dựng danh sách kề, sau đó dùng `shortest_path` để tìm đường từ vị trí hiện tại tới từng khách hàng theo thứ tự GA đã chọn. Hàm `build_route` lấy `current_edge_id` làm điểm bắt đầu, tìm node gần nhất với tọa độ khách hàng và nối các đoạn shortest path lại thành một danh sách `edge_id` liên tục. Cách tách vai trò này giúp GA tập trung vào thứ tự giao khách, còn Dijkstra chịu trách nhiệm tạo đường đi cụ thể trên graph.

Khi tính chi phí cạnh trong graph, hệ thống ưu tiên traffic adapter nếu có. Nếu cạnh đang bị tắc hoặc nằm trong `blocked_edges`, chi phí có thể được xem như vô hạn để tránh chọn cạnh đó. Nếu không có dữ liệu traffic, chi phí được tính từ chiều dài cạnh và tốc độ tham chiếu.

Thành phần graph	Vai trò
<code>GraphNetwork</code>	Load graph từ <code>edges_schema.json</code> và lưu <code>nodes</code> , <code>edges</code> , <code>adjacency</code> .
<code>edge_cost</code>	Tính chi phí cạnh từ traffic state hoặc từ <code>length_meters</code> và <code>max_speed_kmh</code> .
<code>shortest_path</code>	Tìm đường đi ngắn nhất giữa hai node bằng Dijkstra.
<code>nearest_node_to_point</code>	Tìm node gần nhất với tọa độ khách hàng.
<code>build_route</code>	Ghép các đoạn shortest path để tạo <code>new_assigned_route</code> .

Bảng 5.5. Các thành phần sinh tuyến cạnh mới

Nếu quá trình sinh route mới gặp lỗi hoặc không tìm được đường hợp lệ, hệ thống fallback về `old_assigned_route`. Đây là cách xử lý an toàn trong quá trình kiểm thử, vì dashboard vẫn có route hợp lệ để hiển thị thay vì bị mất dữ liệu.

5.8. Lưu kết quả tối ưu vào MongoDB

Sau khi GA chạy xong và tuyến cạnh mới được sinh ra, route optimization service cập nhật kết quả vào MongoDB. Collection được sử dụng là `assigned_routes` trong database `traffic_system`.

Các field quan trọng được ghi gồm `new_assigned_route`, `estimated_total_travel_time`, `optimized_customer_order`, `optimization_result`, `last_optimized_at`, `route_status`, danh sách `customers` và các thông tin phục vụ hiển thị như `total_edges`. Dashboard backend theo dõi collection này bằng MongoDB Change Streams, sau đó phát event `route_optimized` tới frontend.

Field cập nhật	Ý nghĩa
<code>new_assigned_route</code>	Tuyến đường mới ở mức danh sách <code>edge_id</code> .
<code>estimated_total_travel_time</code>	Chi phí hoặc thời gian di chuyển ước lượng sau tối ưu.
<code>optimized_customer_order</code>	Thứ tự giao khách do GA chọn.
<code>optimization_result</code>	Object chi tiết gồm route cũ, route mới, cost, số thể hệ và thuật toán sử dụng.
<code>customers</code>	Danh sách khách hàng kèm thứ tự và trạng thái hiển thị trên dashboard.
<code>total_edges</code>	Số cạnh của route mới, phục vụ panel dashboard và ước lượng phần route còn lại.
<code>last_optimized_at</code>	Thời điểm tối ưu gần nhất dạng epoch milliseconds.
<code>route_status</code>	Trạng thái route, ví dụ <code>optimized</code> .

Bảng 5.6. Các field được cập nhật sau khi tối ưu

Việc lưu kết quả vào MongoDB giúp tách optimizer khỏi dashboard.

Optimizer chỉ cần cập nhật dữ liệu, còn dashboard backend chịu trách nhiệm phát sự kiện gần thời gian thực. Thiết kế này làm hệ thống dễ mở rộng hơn và tránh việc thuật toán tối ưu phải phụ thuộc trực tiếp vào frontend.

```
PS C:\dev\logistics-real-time> kubectl exec -it deployment/mongodb -- mongosh traffic_system --eval 'db.assigned_routes.findOne({}, {vehicle_id:1,new_assigned_route:{$slice:5},customers:{$slice:3},reroute_reason:1,total_edges:1})'
```

```
{
  _id: ObjectId('6a16e6defadc90e84190392a'),
  vehicle_id: 'Truck_001',
  customers: [
    {
      cust_id: 'Cust_T001_1',
      latitude: 21.0263027,
      longitude: 105.8374665,
      order: 1,
      status: 'delivered'
    },
    {
      cust_id: 'Cust_T001_2',
      latitude: 21.0265438,
      longitude: 105.8370144,
      order: 2,
      status: 'delivered'
    },
    {
      cust_id: 'Cust_T001_3',
      latitude: 21.0268901,
      longitude: 105.8363651,
      order: 3,
      status: 'delivered'
    }
  ],
  total_edges: 198,
  new_assigned_route: [
    'E_6658453755_12568521746',
    'E_12568521746_12951168004',
    'E_12951168004_12951168005',
    'E_12951168005_8234749117',
    'E_8234749117_8234749153'
  ],
  reroute_reason: 'Tắc nghẽn tại: E_12357580642_12357580643, E_12357580643_12357580646, E_12357580644_12357580650'
}
```

Hình 5.1. Dữ liệu tuyến đường, khách hàng và lý do reroute trong collection assigned_routes

Hình 5.1 minh họa cấu trúc dữ liệu sau khi optimizer cập nhật route trong MongoDB. Một document route chứa danh sách khách hàng, trạng thái giao hàng, danh sách cạnh đường mới và lý do reroute dựa trên các cạnh tắc nghẽn.

5.9. Worker chạy nền trong Kubernetes

Route optimization được triển khai như một worker chạy nền. Khi container khởi động, worker kết nối tới MongoDB và Redis thông qua service name trong Kubernetes. Sau đó, worker chạy vòng lặp vô hạn, cứ mỗi 5 giây đọc danh sách xe trong collection assigned_routes và gọi optimize_many_vehicles.

Trong lần đầu chạy, nếu một xe chưa có new_assigned_route, hệ thống coi đó là trạng thái cần bootstrap và ép chạy tối ưu một lần. Những lần sau, nếu không có cạnh tắc hoặc cạnh tốc độ thấp trên route, worker bỏ qua để tiết kiệm tài nguyên. Khi có cập nhật mới, log sẽ ghi số xe đã được tính lại tuyến.

Đặc điểm worker	Mô tả
Chu kỳ quét	5 giây.
Nguồn route	MongoDB collection <code>assigned_routes</code> .
Nguồn traffic	Redis, gồm <code>blocked_edges</code> và traffic state theo cạnh.
Điều kiện chạy GA	Route bị ảnh hưởng bởi cạnh tắc hoặc xe chưa có route tối ưu.
Kết quả	Cập nhật route mới và thông tin tối ưu vào MongoDB.

Bảng 5.7. Đặc điểm vận hành của route optimization worker

```
PS C:\dev\Logistics-real-time> kubectl logs deployment/route-optimization --tail=80
🔥 Khởi động Route Optimization Worker...
🔗 Đang kết nối MongoDB: mongodb://mongodb.default.svc.cluster.local:27017/?directConnection=true
🔗 Đang kết nối Redis: redis.default.svc.cluster.local:6379
📄 Đang load bản đồ: /app/data/edges_schema.json
✅ Graph loaded: 5585 nodes, 9026 edges
✅ Đã kết nối thành công! Bắt đầu giám sát giao thông & tối ưu lộ trình 24/7...
INFO:RedisWriter:✅ Redis Connected: redis.default.svc.cluster.local:6379
[CẬP NHẬT] Đã tính toán lại đường đi cho 100 xe do kẹt xe!
```

Hình 5.2. Route optimization worker tải graph đường và cập nhật lại tuyến cho các xe khi có tắc nghẽn

```
PS C:\dev\Logistics-real-time> kubectl exec -it deployment/mongodb -- mongosh traffic_system --eval 'db.assigned_routes.findOne({}, {vehicle_id:1,customers:{$slice:5},new_assigned_route:{$slice:5},route_status:1,last_optimized_at:1})'
{
  "_id": ObjectId("6a16e6defdc98e84190392a"),
  "vehicle_id": "Truck_001",
  "customers": [
    {
      "cust_id": "Cust_T001_1",
      "latitude": 21.0263027,
      "longitude": 105.8374665,
      "order": 1,
      "status": "delivered"
    },
    {
      "cust_id": "Cust_T001_2",
      "latitude": 21.0265438,
      "longitude": 105.8370144,
      "order": 2,
      "status": "delivered"
    },
    {
      "cust_id": "Cust_T001_3",
      "latitude": 21.0268901,
      "longitude": 105.8363651,
      "order": 3,
      "status": "delivered"
    },
    {
      "cust_id": "Cust_T001_0",
      "latitude": 21.0269857,
      "longitude": 105.8361859,
      "order": 4,
      "status": "delivered"
    }
  ]
}
```

Hình 5.3. Thông tin khách hàng và tuyến đường mới được lưu trong MongoDB sau quá trình tối ưu

```

{
  cust_id: 'Cust_T001_5',
  latitude: 21.0270099,
  longitude: 105.8361365,
  order: 5,
  status: 'delivered'
},
last_optimized_at: Long('1779885795441'),
new_assigned_route: [
  'E_6658453755_12568521746',
  'E_12568521746_12951168004',
  'E_12951168004_12951168005',
  'E_12951168005_8234749117',
  'E_8234749117_8234749153'
],
route_status: 'optimized'
}

```

Hình 5.4. Trạng thái tuyến đường được đánh dấu tối ưu cùng thời điểm cập nhật trong MongoDB

Các hình trên kiểm chứng route optimization worker đã tải graph từ `edges_schema.json`, kết nối Redis/MongoDB và ghi lại route mới vào MongoDB. Việc lưu cả danh sách khách hàng, trạng thái giao hàng, tuyến mới và thời điểm tối ưu giúp backend và bot simulator có thể đồng bộ tiến độ tuyến đường trong quá trình chạy.

5.10. Nhận xét và giới hạn

Module route optimization đóng vai trò nối phần xử lý traffic với phần hiển thị tuyến đường. Thay vì chỉ tính toán độc lập trên dữ liệu giả lập, module này được đặt trong vòng lặp dữ liệu của hệ thống: đọc thông tin tuyến đường và khách hàng từ MongoDB, đọc trạng thái giao thông ngắn hạn từ Redis, sau đó ghi kết quả tuyến mới trở lại MongoDB để backend và dashboard có thể nhận được cập nhật.

Trong triển khai hiện tại, thuật toán tối ưu tập trung vào hai nhiệm vụ chính. Thứ nhất, Genetic Algorithm được sử dụng để sắp xếp lại thứ tự giao khách nhằm giảm chi phí di chuyển tổng thể. Thứ hai, sau khi có thứ tự khách hàng, hệ thống sử dụng đồ thị đường đi từ dữ liệu bản đồ để xây dựng danh sách các cạnh đường liên tục cho tuyến mới. Kết quả cuối cùng được lưu dưới dạng `new_assigned_route`, kèm theo thông tin khách hàng, thời gian ước lượng và trạng thái route trong MongoDB.

Tuy nhiên, module này vẫn có một số giới hạn. Trạng thái giao thông từ Redis mới được dùng chủ yếu để phát hiện khi nào cần tối ưu lại tuyến, thay vì tạo ra một mô hình chi phí giao thông phức tạp cho từng cạnh đường. Việc ánh xạ khách hàng hoặc vị trí xe vào đồ thị cũng mới dừng ở mức gần đúng, phù hợp với dữ liệu mô phỏng và bản đồ thử nghiệm. Trong một số trường hợp dữ liệu Redis chưa đầy đủ

hoặc không tìm được đường hợp lệ trên graph, hệ thống ưu tiên giữ route hiện tại hoặc fallback để dashboard vẫn hoạt động ổn định.

Nhìn chung, kết quả đạt được phù hợp với phạm vi bài tập lớn và trọng tâm của môn Big Data. Mục tiêu chính của dự án không phải chứng minh một thuật toán tối ưu toàn cục, mà là xây dựng được vòng lặp dữ liệu gần thời gian thực: Spark tổng hợp traffic state, Redis lưu trạng thái ngắn hạn, route optimizer cập nhật kết quả vào MongoDB và dashboard hiển thị tuyến mới cho người dùng. Đây là nền tảng để hệ thống có thể tiếp tục được cải thiện trong các phiên bản sau.

CHƯƠNG 6. KẾT QUẢ THỬ NGHIỆM VÀ ĐÁNH GIÁ

6.1. Mục tiêu thử nghiệm

Sau khi hoàn thiện các thành phần chính của hệ thống, nhóm tiến hành thử nghiệm để kiểm tra khả năng hoạt động end-to-end của pipeline logistics thời gian thực. Mục tiêu của chương này không phải đo hiệu năng ở quy mô production, mà là xác nhận rằng các service có thể phối hợp đúng vai trò: bot simulator sinh dữ liệu GPS, Kafka truyền sự kiện, Spark xử lý streaming, Redis lưu trạng thái giao thông, MongoDB lưu route assignment và dashboard hiển thị kết quả theo thời gian thực.

Các thử nghiệm được thực hiện trong môi trường cục bộ sử dụng Docker Desktop Kubernetes. Hệ thống được triển khai theo hai nhóm tài nguyên: nhóm hạ tầng gồm Zookeeper, Kafka, Redis và MongoDB; nhóm microservice gồm bot simulator, stream processing, route optimization, dashboard backend và dashboard frontend. Quy trình chạy được thực hiện theo hướng dẫn trong repository: build image, apply hạ tầng, khởi tạo MongoDB replica set, apply microservice, seed route và truy cập dashboard.

6.2. Môi trường triển khai

Môi trường thử nghiệm sử dụng Kubernetes local, phù hợp với phạm vi bài tập lớn. Các service được đóng gói bằng Docker image và kết nối với nhau thông qua Kubernetes Service DNS. Cách triển khai này giúp hệ thống gần với kiến trúc microservice thực tế hơn so với việc chỉ chạy nhiều script rời rạc trên terminal.

Thành phần	Vai trò trong thử nghiệm
Docker Desktop Kubernetes	Cụm Kubernetes local dùng để triển khai các service.
Kafka và Zookeeper	Tiếp nhận và phân phối sự kiện GPS qua topic <code>gps_stream</code> .
Spark Structured Streaming	Đọc Kafka, map-matching GPS, tổng hợp traffic và ghi Redis.
Redis	Lưu traffic state theo key <code>edge:<edge_id></code> và tập <code>blocked_edges</code> .
MongoDB Replica Set	Lưu route assignment, kết quả tối ưu và hỗ trợ Change Streams.
React dashboard	Hiển thị xe, traffic, heatmap và tuyến đường trên bản đồ.

Bảng 6.1. Môi trường và thành phần thử nghiệm

Trong Kubernetes, dashboard frontend được expose qua NodePort hoặc port-forward để truy cập từ trình duyệt. Dashboard backend kết nối tới Kafka, Redis và MongoDB, sau đó phát dữ liệu về frontend bằng Socket.IO. MongoDB cần được khởi tạo replica set để Change Streams hoạt động ổn định.

6.3. Kết quả triển khai các service

Kết quả đầu tiên cần xác nhận là các service có thể khởi động và duy trì trạng thái chạy trong Kubernetes. Khi triển khai thành công, các pod chính của hệ thống đạt trạng thái Running, backend kết nối được tới Redis, MongoDB và Kafka, còn frontend có thể truy cập từ trình duyệt.

Service	Trạng thái mong đợi	Ý nghĩa
bot-simulator	Running	Liên tục sinh dữ liệu GPS và gửi vào Kafka.
stream-processing	Running	Spark job đang đọc Kafka và cập nhật Redis.
route-optimization	Running	Worker định kỳ kiểm tra route và cập nhật MongoDB khi cần.
dashboard-backend	Running	Backend nhận dữ liệu từ Kafka, Redis, MongoDB và phát Socket.IO.
dashboard-frontend	Running	Frontend hiển thị bản đồ và các lớp dữ liệu gần thời gian thực.

Bảng 6.2. Trạng thái mong đợi của các service sau triển khai

Trong quá trình thử nghiệm, một số service có thể cần thời gian khởi động lâu hơn do phải tải package, kết nối broker hoặc chờ MongoDB replica set sẵn sàng. Vì vậy, trạng thái `ContainerCreating` hoặc `retry` ở giai đoạn đầu chưa nhất thiết là lỗi. Điều quan trọng là sau khi các service hạ tầng sẵn sàng, các pod ứng dụng đạt trạng thái ổn định và log không còn xuất hiện exception lặp lại.

6.4. Kiểm chứng luồng dữ liệu GPS

Luồng dữ liệu GPS được kiểm chứng bằng cách quan sát log của bot simulator và dashboard backend. Bot simulator gửi liên tục message vào Kafka topic `gps_stream`. Dashboard backend cũng subscribe cùng topic này để nhận các bản ghi GPS của xe tải, gom thành batch và phát sự kiện `vehicle_batch` về frontend.

Trên frontend, các xe tải được hiển thị bằng marker trên bản đồ. Mỗi marker thể hiện định danh rút gọn và tốc độ hiện tại của xe. Khi dữ liệu Kafka tiếp tục được sinh, vị trí xe trên dashboard được cập nhật theo thời gian. Điều này cho thấy pipeline từ simulator tới Kafka, từ Kafka tới dashboard backend và từ backend tới

frontend đã hoạt động.

Dấu hiệu quan sát	Ý nghĩa
Log bot báo gửi thành công vào <code>gps_stream</code>	Simulator đang tạo dữ liệu GPS và Kafka producer hoạt động.
Dashboard hiển thị marker xe tải	Backend nhận được GPS từ Kafka và phát dữ liệu sang frontend.
Tốc độ và vị trí xe thay đổi theo thời gian	Dữ liệu thời gian thực đang tiếp tục được cập nhật, không chỉ là snapshot tĩnh.
Số lượng xe hiển thị tăng sau khi hệ thống chạy ổn định	Dashboard đã nhận batch dữ liệu từ nhiều phương tiện.

Bảng 6.3. Các dấu hiệu kiểm chứng luồng GPS gần thời gian thực

Kết quả này xác nhận phần ingestion và hiển thị gần thời gian thực của hệ thống đạt yêu cầu cơ bản. Đây là điều kiện nền để tiếp tục kiểm chứng các tầng xử lý traffic và tối ưu tuyến đường.

6.5. Kiểm chứng xử lý Spark và Redis

Tầng Spark được kiểm chứng thông qua log của deployment stream-processing và dữ liệu trong Redis. Khi Spark job chạy thành công, log cho thấy ứng dụng Structured Streaming khởi động, đọc dữ liệu từ Kafka, thực hiện map-matching và xử lý micro-batch.

Kết quả của Spark không được ghi trực tiếp lên dashboard dưới dạng bản ghi GPS thô, mà được chuyển thành traffic state theo từng cạnh đường. Mỗi cạnh có dữ liệu được lưu trong Redis với key dạng `edge:<edge_id>`. Dashboard backend định kỳ đọc các key này, parse JSON payload và phát sự kiện `traffic_batch` tới frontend.

Kết quả cần kiểm chứng	Cách quan sát
Spark đọc được Kafka	Log stream processing không báo lỗi Kafka timeout hoặc schema parse lặp lại.
Map-matching hoạt động	Redis xuất hiện key <code>edge:<edge_id></code> thay vì chỉ có UNKNOWN.
Traffic state được cập nhật	Các field như <code>avg_speed</code> , <code>vehicle_count</code> và <code>is_congested</code> thay đổi theo batch.
Dashboard nhận traffic	Các cạnh đường trên bản đồ đổi màu theo tốc độ trung bình.
Heatmap hoạt động	Các vùng có tốc độ thấp tạo điểm nóng trên bản đồ.

Bảng 6.4. Kiểm chứng kết quả xử lý Spark và Redis

Khi Redis có dữ liệu traffic, frontend vẽ lại các cạnh đường trên bản đồ với màu sắc phản ánh vận tốc trung bình hiện tại. Những cạnh chưa có dữ liệu vẫn sử dụng màu mặc định của bản đồ. Theo logic hiển thị hiện tại, cạnh có vận tốc lớn hơn 20 km/h được tô màu xanh, cạnh có vận tốc từ trên 10 km/h đến 20 km/h được tô màu cam, và cạnh có vận tốc nhỏ hơn hoặc bằng 10 km/h được tô màu đỏ. Cơ chế này giúp quan sát nhanh trạng thái giao thông từ dữ liệu streaming.

Về ý nghĩa kết quả, Spark không chỉ chuyển tiếp dữ liệu GPS thô mà còn biến các sự kiện rời rạc thành trạng thái giao thông theo cạnh đường. Đây là bước xử lý quan trọng nhất của pipeline Big Data: dữ liệu vị trí liên tục được gom theo thời gian, map-matching lên graph đường và tổng hợp thành thông tin có thể phục vụ dashboard cũng như các quyết định route update.

6.6. Kiểm chứng route optimization

Route optimization được kiểm chứng bằng cách seed dữ liệu xe tải vào MongoDB, sau đó để worker route-optimization chạy nền. Worker đọc collection `assigned_routes`, sử dụng route assignment cùng các tín hiệu traffic từ Redis, đặc biệt là tập `blocked_edges`, để xác định khi nào cần tái tối ưu và cập nhật kết quả route vào MongoDB.

Sau khi optimizer cập nhật MongoDB, dashboard backend sử dụng MongoDB Change Streams để phát sự kiện `route_optimized`. Frontend nhận sự kiện này và cập nhật danh sách route theo từng xe. Khi người dùng chọn một xe trên dashboard, hệ thống hiển thị tuyến đường ở dạng polyline, đồng thời dùng vị trí hiện tại của xe để ước lượng phần route còn lại và ETA còn lại.

Kết quả cần kiểm chứng	Dấu hiệu đạt được
MongoDB có route assignment	Collection <code>assigned_routes</code> chứa xe, tuyến hiện tại và danh sách khách còn lại.
Optimizer chạy nền	Log worker cho thấy quá trình quét xe và cập nhật route khi cần.
MongoDB được cập nhật	Document có thêm <code>new_assigned_route</code> , <code>optimized_customer_order</code> , <code>customers</code> và <code>route_status</code> .
Dashboard nhận route mới	Panel bên trái xuất hiện danh sách xe có lộ trình tối ưu.
Map hiển thị polyline	Khi chọn xe, dashboard vẽ tuyến đường theo danh sách <code>edge_id</code> .
ETA và cạnh còn lại	Frontend so khớp vị trí xe với các cạnh trong route để ước lượng phần đường còn lại.

Bảng 6.5. Kiểm chứng kết quả route optimization

Kết quả này cho thấy vòng lặp route update đã được nối vào pipeline chính. Optimizer không cần giao tiếp trực tiếp với frontend; việc cập nhật MongoDB là đủ để dashboard backend nhận thay đổi và phát sự kiện gần thời gian thực. Trong phạm vi hiện tại, GA đóng vai trò tối ưu thứ tự giao khách, còn bước Dijkstra/graph route building chuyển thứ tự đó thành danh sách cạnh đường để dashboard có thể hiển thị.

Cách đánh giá này phù hợp với phạm vi môn Big Data. Thành phần route optimization được xem như một module tiêu thụ dữ liệu đã xử lý và phản hồi kết quả vào hệ thống lưu trữ, thay vì một bài toán thuật toán độc lập cần chứng minh nghiệm tối ưu toàn cục. Điều quan trọng trong thử nghiệm là MongoDB, Redis,

worker tối ưu, backend Change Streams và dashboard đã tạo thành một vòng lặp cập nhật có thể quan sát được.

6.7. Kết quả hiển thị trên dashboard

Dashboard là nơi tổng hợp kết quả của toàn bộ pipeline. Frontend sử dụng React, Leaflet và Socket.IO để hiển thị nhiều lớp dữ liệu trên cùng một bản đồ: marker xe tải, màu cạnh đường theo traffic, heatmap tắc nghẽn, tuyến đường của từng xe và thông tin ETA/cạnh còn lại.

Các thành phần chính trên dashboard gồm:

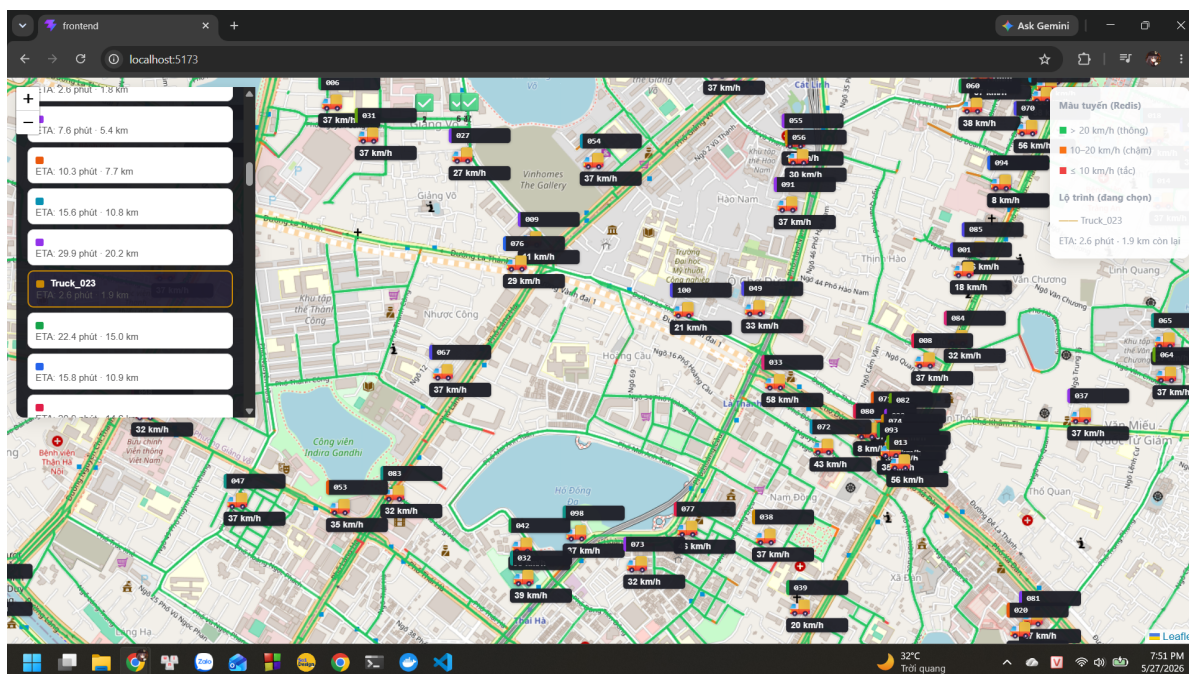
- Bản đồ nền OpenStreetMap.
- Marker xe tải hiển thị định danh và tốc độ.
- Lớp cạnh đường, trong đó màu cạnh phản ánh tốc độ trung bình từ Redis.
- Heatmap thể hiện vùng có mật độ tắc nghẽn hoặc tốc độ thấp.
- Panel danh sách xe, tuyến tối ưu, ETA và số cạnh còn lại.
- Polyline hiển thị tuyến đường của xe khi người dùng chọn một xe.
- Logic ước lượng phần route còn lại bằng cách tìm cạnh trong route gần nhất với vị trí xe hiện tại.

Dữ liệu hiển thị	Nguồn dữ liệu
Vị trí xe	Kafka topic <code>gps_stream</code> , qua dashboard backend.
Tốc độ theo cạnh	Redis key <code>edge:<edge_id></code> .
Heatmap	Traffic batch được tạo từ Redis.
Route snapshot	MongoDB collection <code>assigned_routes</code> .
Route optimized	MongoDB Change Streams và Socket.IO event <code>route_optimized</code> .
ETA/cạnh còn lại	Vị trí xe từ Kafka kết hợp với route snapshot trên frontend.

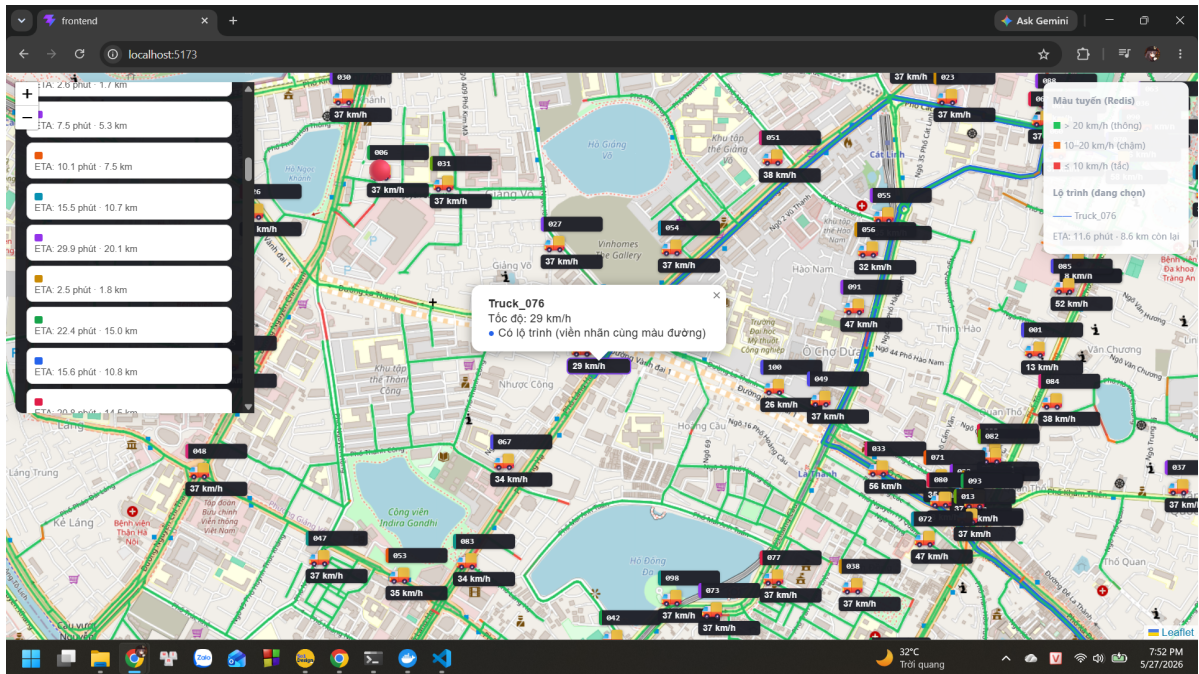
Bảng 6.6. Nguồn dữ liệu cho các lớp hiển thị trên dashboard

Về mặt trải nghiệm, dashboard giúp kiểm chứng trực quan rằng dữ liệu không chỉ nằm trong log hoặc database, mà đã được đưa tới giao diện người dùng. Khi

xe di chuyển, marker cập nhật; khi Redis có traffic state, cạnh đường đổi màu; khi MongoDB có route mới, panel route và polyline được cập nhật. Ngoài ra, frontend còn ước lượng cạnh gần nhất của xe trong route để hiển thị ETA và số cạnh còn lại, giúp người dùng quan sát tiến độ giao hàng trực quan hơn.



Hình 6.1. Dashboard hiển thị tuyến đường được chọn cùng ETA và quãng đường còn lại



Hình 6.2. Thông tin phương tiện và tuyến đường hiển thị trực tiếp trên bản đồ

Hình 6.1 và Hình 6.2 thể hiện kết quả trực quan hóa cuối cùng: dashboard hiển thị nhiều xe trên bản đồ, lớp màu giao thông theo tốc độ trung bình, tuyến đường được chọn, ETA và quãng đường còn lại. Khi người dùng chọn một xe, frontend hiển thị thêm thông tin tốc độ và trạng thái có route trực tiếp trên bản đồ.

6.8. Đánh giá mức độ hoàn thành

Dựa trên kết quả thử nghiệm, hệ thống đã hoàn thiện được luồng xử lý chính của một bài toán logistics thời gian thực theo định hướng Big Data. Dữ liệu GPS được sinh liên tục, truyền qua Kafka, xử lý bằng Spark Structured Streaming, cập nhật trạng thái giao thông vào Redis, đồng bộ thông tin tuyến đường trong MongoDB và hiển thị trực quan trên dashboard. Các thành phần này không hoạt động rời rạc mà được kết nối thành một pipeline end-to-end, phản ánh đúng trọng tâm của môn học là thu thập, xử lý, lưu trữ và khai thác dữ liệu theo luồng.

Mục tiêu	Trạng thái	Ghi chú
Sinh dữ liệu GPS mô phỏng	Đạt	Bot simulator gửi dữ liệu vào Kafka.
Truyền dữ liệu qua Kafka	Đạt	Kafka topic <code>gps_stream</code> được dùng bởi Spark và dashboard backend.
Xử lý streaming bằng Spark	Đạt	Spark parse GPS, map-matching, aggregate theo window và ghi Redis.
Lưu traffic state ngắn hạn	Đạt	Redis lưu key theo cạnh đường và tập <code>blocked_edges</code> .
Tối ưu tuyến đường	Đáp ứng mục tiêu tích hợp	GA sử dụng route assignment và tín hiệu <code>blocked_edges</code> để hỗ trợ tái tối ưu thứ tự giao khách; Dijkstra/graph route building sinh danh sách cạnh đường và cập nhật kết quả route vào MongoDB.
Hiển thị dashboard gần thời gian thực	Đạt	Frontend hiển thị xe, traffic, heatmap, route, ETA và số cạnh còn lại.
Triển khai bằng Kubernetes	Đạt	Các thành phần được tách thành deployment và service riêng.

Bảng 6.7. Đánh giá mức độ hoàn thành các mục tiêu

Kết quả này phù hợp với phạm vi của một bài tập lớn môn Big Data, vì hệ thống đã thể hiện được đầy đủ vòng đời dữ liệu từ lúc sinh sự kiện GPS đến khi trạng thái giao thông và tuyến đường được hiển thị trên bản đồ. Các thành phần được tách theo đúng vai trò, cho phép quan sát rõ từng tầng của pipeline: ingestion, stream processing, state storage, route update và visualization gần thời gian thực.

6.9. Giới hạn trong thử nghiệm

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số giới hạn cần ghi nhận. Các giới hạn này chủ yếu đến từ phạm vi bài tập lớn và môi trường triển khai cục bộ, đồng thời cũng là cơ sở để xác định các hướng mở rộng nếu hệ thống được phát triển ở quy mô lớn hơn.

Thứ nhất, dữ liệu GPS và traffic là dữ liệu mô phỏng, chưa tích hợp trực tiếp với nguồn dữ liệu giao thông thực tế. Vì vậy, kết quả traffic phản ánh logic mô phỏng và pipeline xử lý, không phải tình hình giao thông thật ngoài đời.

Thứ hai, hệ thống chạy trên Kubernetes local nên tài nguyên CPU, RAM và storage còn hạn chế. Khi số lượng xe, tần suất gửi message hoặc kích thước bản đồ tăng mạnh, cần có cấu hình tài nguyên và scaling nghiêm túc hơn.

Thứ ba, cấu hình lưu trữ hiện được lựa chọn để phù hợp với môi trường chạy cục bộ và quá trình kiểm thử. Checkpoint và Parquet sink của Spark dùng đường dẫn tạm, Redis traffic state có TTL ngắn, còn MongoDB chạy trong môi trường local. Nếu mở rộng hệ thống cho quá trình vận hành dài hạn, cần bổ sung persistent volume, monitoring, backup và cơ chế quản lý tài nguyên đầy đủ hơn.

Thứ tư, thuật toán tối ưu tuyến đường được triển khai ở mức phù hợp với trọng tâm của môn Big Data. Genetic Algorithm trong hệ thống có nhiệm vụ tối ưu thứ tự giao khách, sử dụng route assignment và các tín hiệu traffic như `blocked_edges` để hỗ trợ cập nhật route; Dijkstra/graph route building đảm nhiệm việc sinh đường đi cụ thể ở mức `edge_id`. Trong các hướng phát triển tiếp theo, fitness function, adapter đọc Redis và mô hình chi phí đường đi có thể tiếp tục được chuẩn hóa để khai thác trực tiếp các trường như `avg_speed` và `estimated_travel_time` trên từng cạnh đường.

6.10. Kết luận chương

Các thử nghiệm cho thấy hệ thống đã vận hành được như một pipeline logistics thời gian thực hoàn chỉnh trong phạm vi bài tập lớn. Dữ liệu GPS được sinh liên tục, truyền qua Kafka, xử lý bằng Spark, lưu trạng thái vào Redis, kết hợp với MongoDB để hỗ trợ tối ưu tuyến đường và cuối cùng được hiển thị trên dashboard.

Kết quả quan trọng nhất không chỉ là từng service chạy riêng lẻ, mà là các service đã được nối thành một chuỗi xử lý end-to-end. Điều này cho thấy hướng tiếp cận của nhóm phù hợp với mục tiêu của môn Big Data: xây dựng một hệ thống có khả năng tiếp nhận dữ liệu liên tục, xử lý theo luồng, duy trì trạng thái phục vụ truy vấn nhanh và sử dụng kết quả xử lý để hỗ trợ quyết định vận hành.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Tổng kết dự án

Dự án **Hệ thống điều phối vận tải thời gian thực** đã xây dựng được một hệ thống giám sát và hỗ trợ tối ưu hóa logistics gần thời gian thực theo định hướng Big Data. Thay vì chỉ mô phỏng một thuật toán đơn lẻ, hệ thống được triển khai như một pipeline dữ liệu lớn end-to-end, trong đó dữ liệu GPS được sinh liên tục, truyền qua Kafka, xử lý bằng Spark Structured Streaming, lưu trạng thái giao thông vào Redis, cập nhật kết quả tối ưu trong MongoDB và hiển thị trên dashboard gần thời gian thực.

Kết quả quan trọng nhất của dự án là chứng minh được khả năng kết nối nhiều thành phần độc lập thành một chuỗi xử lý thống nhất. Bot simulator tạo dữ liệu GPS cho nhiều phương tiện; Kafka đóng vai trò message broker; Spark thực hiện map-matching và tổng hợp traffic state; Redis lưu trạng thái ngắn hạn theo từng cạnh đường và tập `blocked_edges`; route optimization worker sử dụng route assignment cùng tín hiệu ùn tắc để cập nhật route; dashboard backend và frontend trực quan hóa trạng thái hệ thống, route, ETA và tiến độ còn lại của từng xe.

Về mặt triển khai, hệ thống đã được container hóa và chạy bằng Kubernetes. Điều này giúp dự án không chỉ dừng ở mức chạy script cục bộ, mà tiến gần hơn tới mô hình triển khai thực tế của các hệ thống dữ liệu lớn hiện đại, nơi mỗi thành phần có trách nhiệm riêng và giao tiếp với nhau thông qua service, message broker hoặc database.

7.2. Các kết quả đạt được

Trong phạm vi bài tập lớn, nhóm đã hoàn thành các mục tiêu chính đặt ra ban đầu. Hệ thống có thể sinh dữ liệu GPS, xử lý streaming, lưu trạng thái giao thông, hỗ trợ tối ưu tuyến đường dựa trên traffic state và hiển thị kết quả trên dashboard.

Mục tiêu	Kết quả đạt được
Mô phỏng dữ liệu GPS	Bot simulator sinh dữ liệu vị trí, tốc độ và thời gian của phương tiện, sau đó gửi vào Kafka topic <code>gps_stream</code> .
Xây dựng pipeline streaming	Spark Structured Streaming đọc dữ liệu từ Kafka, parse JSON, map-matching tọa độ vào cạnh đường và aggregate traffic theo window.
Lưu trạng thái giao thông	Redis lưu traffic state theo key <code>edge:<edge_id></code> và duy trì tập <code>blocked_edges</code> .
Tối ưu tuyến đường	Route optimization worker đọc route assignment từ MongoDB, sử dụng tín hiệu <code>blocked_edges</code> , chạy Genetic Algorithm để tối ưu thứ tự giao khách, dùng graph route building/Dijkstra để sinh route cạnh và cập nhật kết quả route.
Trực quan hóa gần thời gian thực	Dashboard hiển thị vị trí xe, tốc độ theo cạnh đường, heatmap, tuyến đường, ETA và số cạnh còn lại trên bản đồ.
Triển khai bằng Kubernetes	Các thành phần được đóng gói thành Docker image và triển khai bằng Kubernetes YAML.

Bảng 7.1. Tổng hợp các kết quả đạt được của dự án

Bên cạnh việc hoàn thành các chức năng chính, dự án còn giúp nhóm hiểu rõ hơn về cách các công nghệ dữ liệu lớn phối hợp trong một hệ thống thực tế. Kafka không chỉ là nơi truyền message, Spark không chỉ là công cụ tính toán, Redis không chỉ là cache, và MongoDB không chỉ là database lưu kết quả. Khi được kết nối đúng vai trò, mỗi thành phần trở thành một mắt xích trong vòng lặp dữ liệu gần thời gian thực.

7.3. Bài học kinh nghiệm

Quá trình phát triển dự án cho thấy việc xây dựng một hệ thống dữ liệu lớn không chỉ khó ở thuật toán, mà còn khó ở tích hợp. Một module chạy đúng khi test riêng chưa chắc hoạt động ổn định khi đặt vào pipeline có Kafka, Spark, Redis,

MongoDB, Socket.IO và Kubernetes.

Bài học đầu tiên là cần thiết kế schema dữ liệu rõ ràng ngay từ đầu. Các field như `entity_id`, `vehicle_id`, `edge_id`, `assigned_route` và `new_assigned_route` xuất hiện ở nhiều module khác nhau. Nếu tên field không thống nhất, hệ thống rất dễ phát sinh lỗi khó debug. Vì vậy, việc chuẩn hóa input trong `optimizer_input_adapter` và mô tả schema trong báo cáo là cần thiết.

Bài học thứ hai là version của các framework có ảnh hưởng lớn tới độ ổn định. Spark, PySpark, Kafka connector, Java runtime và Python package cần tương thích với nhau. Một thay đổi nhỏ trong version có thể làm service build lâu hơn, crash khi chạy hoặc sinh lỗi runtime khó phát hiện. Do đó, các dependency quan trọng cần được pin version rõ ràng trong Dockerfile.

Bài học thứ ba là Kubernetes giúp hệ thống giống môi trường triển khai thật hơn, nhưng cũng làm quá trình debug phức tạp hơn. Khi service lỗi, cần biết kiểm tra pod status, log deployment, biến môi trường, service DNS, port-forward và trạng thái của các service phụ thuộc. Đây là kỹ năng quan trọng khi làm việc với các hệ thống phân tán.

Bài học cuối cùng là với bài toán gần thời gian thực, giao diện chỉ là phần nổi. Để dashboard hiển thị đúng, toàn bộ pipeline phía sau phải hoạt động: dữ liệu phải vào Kafka, Spark phải xử lý được, Redis phải có state, MongoDB phải có route và backend phải phát sự kiện đúng format. Ngoài ra, frontend cũng cần xử lý hợp lý mối quan hệ giữa vị trí xe hiện tại và route đã lưu để hiển thị ETA hoặc phần đường còn lại. Chính vì vậy, debug hệ thống cần đi theo luồng dữ liệu, không chỉ nhìn vào frontend.

7.4. Hạn chế của hệ thống

Mặc dù đã hoàn thành các mục tiêu chính trong phạm vi bài tập lớn, hệ thống vẫn còn một số hạn chế cần được nhìn nhận rõ. Các hạn chế này không phủ nhận kết quả đạt được, mà giúp xác định những điểm cần cải thiện nếu hệ thống được phát triển ở quy mô lớn hơn hoặc tích hợp với dữ liệu thực tế.

Thứ nhất, dữ liệu GPS và traffic trong dự án là dữ liệu mô phỏng. Cách làm

này phù hợp với phạm vi bài tập lớn vì giúp chủ động tạo nhiều phương tiện, nhiều tuyến đường và nhiều trạng thái giao thông khác nhau. Tuy nhiên, dữ liệu mô phỏng chưa thể phản ánh đầy đủ các yếu tố của dữ liệu thực tế như sai số GPS, mất tín hiệu, tốc độ thay đổi bất thường hoặc điều kiện đường thay đổi ngoài dự kiến.

Thứ hai, pipeline streaming hiện tại sử dụng trường `edge_id` được gửi kèm trong message GPS để tổng hợp traffic state theo cạnh đường. Cách này giúp Spark xử lý nhanh hơn và tránh phải thực hiện map-matching nặng trong mỗi micro-batch. Tuy nhiên, nếu hệ thống chuyển sang dùng GPS thật, cần bổ sung bước map-matching chặt chẽ hơn để xác định phương tiện đang nằm trên cạnh đường nào.

Thứ ba, traffic state hiện được tổng hợp bằng các chỉ số đơn giản từ dữ liệu streaming, gồm vận tốc trung bình, số lượng phương tiện trong window, thời gian di chuyển ước lượng và trạng thái tắc nghẽn theo ngưỡng vận tốc. Cách tiếp cận này dễ hiểu, dễ kiểm thử và phù hợp với hệ thống demo, nhưng vẫn chỉ phản ánh trạng thái giao thông quan sát được trong cửa sổ xử lý gần nhất.

Thứ tư, module tối ưu tuyến đường đã được tích hợp vào vòng lặp dữ liệu của hệ thống nhưng vẫn ở mức phù hợp với phạm vi bài tập lớn. Route optimizer đọc dữ liệu tuyến đường từ MongoDB và trạng thái giao thông từ Redis để quyết định khi nào cần cập nhật tuyến. Kết quả tối ưu được ghi lại vào MongoDB để backend và dashboard nhận được cập nhật. Tuy nhiên, hệ thống chưa hướng tới việc chứng minh tuyến đường là tối ưu tuyệt đối trong mọi trường hợp, mà tập trung vào việc tạo được một pipeline cập nhật route hoạt động được từ đầu đến cuối.

Thứ năm, môi trường triển khai hiện tại là Kubernetes local trên máy cá nhân. Hệ thống đã tách được các thành phần thành service riêng và chạy được end-to-end, nhưng chưa có đầy đủ các cơ chế của môi trường production như autoscaling, monitoring tập trung, persistent volume cho toàn bộ dữ liệu quan trọng, backup định kỳ và recovery khi service lỗi. Nếu triển khai trên cụm thật, các thành phần này cần được bổ sung để tăng độ ổn định của hệ thống.

7.5. Hướng phát triển

Từ nền tảng hiện tại, hệ thống có thể được phát triển theo nhiều hướng để tiến gần hơn tới một giải pháp logistics gần thời gian thực trong thực tế. Các hướng phát triển này bao gồm cải thiện dữ liệu, thuật toán, hạ tầng và trải nghiệm dashboard.

Hướng phát triển	Mô tả
Tích hợp dữ liệu thực	Kết nối với nguồn GPS thật, API bản đồ hoặc nguồn dữ liệu giao thông công cộng để giảm phụ thuộc vào dữ liệu mô phỏng.
Cải thiện map-matching	Bổ sung cơ chế kiểm tra theo hướng di chuyển, khoảng cách tới cạnh đường và độ ổn định giữa các bản ghi GPS liên tiếp, thay vì chỉ dựa trên cạnh gần nhất.
Mở rộng kịch bản thử nghiệm	Thêm một số kịch bản kiểm thử như tăng số lượng phương tiện, tạo cạnh đường có tốc độ thấp hoặc mô phỏng cạnh đường bị chặn tạm thời để đánh giá phản ứng của hệ thống.
Cải thiện cơ chế cập nhật tuyến	Sử dụng rõ hơn vận tốc trung bình theo từng cạnh trong Redis để phát hiện tuyến đang đi qua đoạn đường bất lợi, đồng thời hạn chế reroute quá thường xuyên để hệ thống ổn định hơn.
Tăng độ bền hạ tầng	Bổ sung persistent volume, health check, logging tập trung, backup dữ liệu và dashboard theo dõi trạng thái các service khi chạy trên Kubernetes.
Cải thiện dashboard	Thêm bộ lọc xe, thống kê tổng quan, cảnh báo tắc nghẽn, lịch sử thay đổi route và khả năng replay dữ liệu trong một khoảng thời gian thử nghiệm.

Bảng 7.2. Một số hướng phát triển tiếp theo

Trong ngắn hạn, hướng cải thiện có tác động lớn nhất là làm chặt hơn liên kết giữa traffic state và route optimization. Cụ thể, fitness function có thể sử dụng trực tiếp chi phí đi qua từng cạnh dựa trên `estimated_travel_time` trong Redis, thay vì chỉ dùng xấp xỉ khoảng cách và tốc độ trung bình. Khi đó, route mới sẽ phản ánh traffic gần thời gian thực rõ ràng hơn.

Trong dài hạn, hệ thống có thể mở rộng thành một nền tảng thử nghiệm cho

bài toán Vehicle Routing Problem theo thời gian thực. Khi số lượng xe, số lượng khách hàng và dữ liệu giao thông tăng lên, hệ thống cần cơ chế phân vùng dữ liệu, scale Spark, scale Kafka consumer, tối ưu Redis access và tách route optimization thành nhiều worker song song.

7.6. Kết luận chung

Dự án đã hoàn thành mục tiêu xây dựng một hệ thống logistics gần thời gian thực có đủ các tầng quan trọng của một pipeline dữ liệu lớn: ingestion, streaming processing, state storage, route optimization, backend gần thời gian thực, frontend visualization và deployment bằng Kubernetes.

Điểm nổi bật của hệ thống là các thành phần không tồn tại rời rạc, mà được kết nối thành một vòng lặp dữ liệu: phương tiện sinh GPS, Spark biến GPS thành traffic state, Redis duy trì trạng thái ngắn hạn và tập `blocked_edges`, optimizer dùng route assignment cùng tín hiệu ùn tắc để cập nhật route, dashboard hiển thị thay đổi và tiến độ còn lại cho người dùng. Vòng lặp này thể hiện đúng tinh thần của các hệ thống dữ liệu lớn hiện đại, nơi giá trị không nằm ở việc lưu trữ dữ liệu đơn thuần, mà nằm ở khả năng phản ứng nhanh với sự thay đổi của dữ liệu.

Dù còn nhiều giới hạn, dự án đã tạo được nền tảng kỹ thuật rõ ràng để phát triển tiếp. Với các cải tiến về dữ liệu thực, thuật toán map-matching, mô hình traffic, route optimization và hạ tầng triển khai, hệ thống có thể được mở rộng thành một bài toán nghiên cứu và ứng dụng sâu hơn trong lĩnh vực logistics thông minh.